LLM 기반 Triton GPU 커널 자동 생성의 체계적 평가

Anonymous Author(s)

Abstract

GPU 커널 최적화는 고성능 컴퓨팅과 딥러닝 워크로드의 핵심 과제이며, OpenAI Triton은 Python 수준의 추상화로 GPU 프로그래밍의 접근성을 높였다. 본 연구는 대규모 언어 모델(LLM)을 활용한 Triton GPU 커널 자동 생성의 가능성과 한계를 체계적으로 평가한다. KernelBench 벤치마크의 250개 문제(Level 1–3)를 대상으로, 상용 모델 7종(gpt-4o-mini, gpt-4o, gpt-5.2, gpt-5.2-pro, gpt-5.4-nano, gpt-5.5, claude-opus-4-7)과 오픈소스 모델 1종(Qwen 2.5 Coder 32B)의 Triton 커널 생성 능력을 비교하였다. 에러 피드백 기반 반복적 개선(iterative refinement) 파이프라인을 설계하고, 다중 모델 캐스케이딩(cascading) 전략을 제안하여 5개 모델 합집합으로 51/100(51%)의 Level 1 정답률을 달성하였다. 실험 결과, claude-opus-4-7이 단일 모델 최고 성능(46/100, 46%)을 기록하였으며, gpt-5.5와 claude-opus-4-7이 처음 44문제에서 동일한 37개를 해결하는 플래그십 수렴 현상(Flagship Convergence)을 발견하였다. 에러 분석을 통해 컴파일 에러율이 모델 능력의 유효한 프록시(proxy)이고 값 불일치(value mismatch)가 정확도 향상의 핵심 병목임을 밝혔다. 반복 횟수 ablation 연구에서는 반복의 깊이보다 생성의 다양성이 더 효과적임을 보였다.

CCS Concepts: • **Software and its engineering → Just-in-time compilers**; **Software performance**; • **Computing methodologies → Machine learning approaches**.

Keywords: GPU kernel generation, Large language models, Triton, Code generation, KernelBench

ACM Reference Format:

Anonymous Author(s). 2027. LLM 기반 Triton GPU 커널 자동 생성의 체계적 평가. In *Proceedings of International Symposium on Code Generation and Optimization (CGO '27).* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 서론

1.1 GPU 커널 최적화와 Triton의 등장

GPU 가속 컴퓨팅은 딥러닝, 과학 시뮬레이션, 대규모 데이터 처리 등 현대 컴퓨팅의 핵심 인프라로 자리잡았다. 특히 대규모 언어 모델(LLM)의 학습과 추론에서 GPU 커널의 성능은 전체 시스템 효율성을 좌우하는 결정적 요인이다 [12]. 그러나 고성능 GPU 커널을 작성하는 것은 메모리 계층 구조, 스레드 동기화, 워프(warp) 수준의 최적화 등 하드웨어에 대한 깊은 이해를 요구하는 전문적인 작업이다 [25]. 전통적으로 CUDA C/C++가 GPU 프로그래밍의 사실상 표준으로 사용되어 왔으나, 그 복잡성은 GPU 최적화의 보편적 접근을 어렵게 하는 주요 장벽이었다.

CGO '27,
2027. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

OpenAI Triton [50]은 이러한 문제를 해결하기 위해 제안된 Python 기반 GPU 프로그래밍 언어이다. Triton은 타일(tile) 기반 프로그래밍 모델을 통해 메모리 접근 패턴, 공유 메모리 관리, 스레드 스케줄링 등의 저수준 최적화를 컴파일러에 위임함으로써, 개발자가 알고리즘 로직에 집중할 수 있도록 한다. PyTorch 2.0 이후 `torch.compile`의 기본 백엔드로 채택되면서 Triton의 중요성은 더욱 증대되고 있다 [2]. 그럼에도 불구하고, 효율적인 Triton 커널을 작성하기 위해서는 타일 크기 선정, 메모리 coalescing 패턴, reduction 전략 등에 대한 상당한 전문 지식이 여전히 필요하다.

1.2 LLM 코드 생성 능력의 발전

대규모 언어 모델은 최근 수 년간 코드 생성 능력에서 괄목할 만한 발전을 이루었다. Codex [5]로부터 시작하여 CodeGen [37], StarCoder [31], GPT-4 [38]에 이르기까지, LLM은 자연어 명세로부터 다양한 프로그래밍 언어의 코드를 생성하는 능력을 지속적으로 향상시켜 왔다. HumanEval [5], MBPP [3], SWE-bench [23] 등의 벤치마크에서 최신 모델들은 전문 개발자에 필적하는 성능을 보이고 있다. 특히 GPT-5.2와 같은 최신 세대 모델은 복잡한 알고리즘 구현과 다중 파일에 걸친 코드 수정 능력에서 상당한 진전을 보여주고 있다.

그러나 이러한 LLM의 코드 생성 능력이 GPU 커널이라는 특수한 도메인에서 어느 정도 유효한지는 아직 충분히 규명되지 않았다. GPU 커널 코드는 일반적인 소프트웨어 코드와 달리 하드웨어 아키텍처에 대한 깊은 이해, 병렬 처리 패턴, 메모리 최적화 전략 등 도메인 특화 지식을 요구한다. 또한 생성된 커널의 평가는 단순한 기능적 정확성(correctness)을 넘어 성능(performance) 측면까지 고려해야 하므로, 일반적인 코드 생성 벤치마크와는 본질적으로 다른 평가 체계를 필요로 한다.

1.3 연구 문제

본 연구는 다음과 같은 핵심 연구 질문을 설정한다: *현재의 LLM은 PyTorch 레퍼런스 구현으로부터 기능적으로 정확한 (functionally correct) Triton GPU 커널을 얼마나 효과적으로 생성할 수 있는가?*

이 질문에 답하기 위해 우리는 KernelBench [40]를 평가 프레임워크로 채택하였다. KernelBench는 PyTorch로 작성된 GPU 연산의 레퍼런스 구현을 등가의 Triton 커널로 변환하는 과제를 정의하며, 난이도에 따라 Level 1(단일 연산, 100문제), Level 2(연산 조합, 100문제), Level 3(전체 아키텍처, 50문제)의 총 250개 문제를 포함한다. 각 문제는 레퍼런스 PyTorch 구현과 테스트 인프라를 제공하므로, 생성된 커널의 정확성을 자동으로 검증할 수 있다.

1.4 기존 연구의 한계

LLM 기반 GPU 커널 생성에 관한 기존 연구들은 몇 가지 한계를 지닌다. 첫째, KernelBench의 원 논문 [40]은 당시 사용 가능한 모델에 대한 기본적인 평가만을 제시하였으며, 최신 세대 모델(GPT-5.2 등)과 오픈소스 모델에 대한 비교가 부재하다. 둘째, 단일 모델의 성능 한계를 극복하기 위한 다중 모델 전략에 대한 체계적인 분석이 수행되지 않았다. 셋째, 반복적 개선 전략에서 반복 횟수와 성능 간의 관계, 즉 더 많은 반복이 더 나은 결과를 보장하는지에 대한 실증적 분석이 부족하다. 넷째, 추론(reasoning) 특화 모델이 코드 생성 과제에서 표준 모델 대비 어떤 특성을 보이는지에 대한 분석이 제한적이다. 마지막으로, 생성 실패의 원인을 체계적으로 분류하고 모델 간 실패 패턴의 차이를 분석한 연구가 부족하다.

1.5 연구 기여

본 연구의 주요 기여는 다음 네 가지로 요약된다.

1. **다중 LLM 모델의 체계적 비교 평가.** 상용 모델 7종(gpt-4o-mini, gpt-4o, gpt-5.2, gpt-5.2-pro, gpt-5.4-nano, gpt-5.5, claude-opus-4-7)과 오픈소스 모델 1종(Qwen 2.5 Coder 32B)에 대해 체계적 비교 평가를 수행하였다. claude-opus-4-7이 단일 모델 기준 최고 성능(46/100, 46%)을 달성하였으며, 추론 특화 모델(gpt-5.2-pro)은 코드 생성에 비효율적임을 확인하였다.
2. **다중 모델 캐스케이딩 전략 제안 및 검증.** 개별 모델의 정답 집합이 상이한 점에 착안하여, 다수의 모델을 순차적으로 적용하는 캐스케이딩 전략을 제안하였다. 5개 모델의 합집합으로 Level 1에서 51/100(51%)을 달성하였으며, 플래그십 수렴 현상(Flagship Convergence)을 발견하였다.
3. **반복 횟수 ablation 연구를 통한 생성 전략 분석.** 반복 횟수를 1, 3, 5, 10회로 변화시키며 성능 변화를 측정한 결과, 개별 실행의 최고 성능(22개)과 전체 합산 성능(28개) 사이에 유의미한 차이가 있음을 발견하였다. 이는 반복의 깊이(depth)보다 생성의 다양성(diversity)이 최종 성능 향상에 더 효과적임을 시사한다.
4. **에러 분류 체계 기반 실패 모드 분석.** 생성 실패를 컴파일 에러, 값 불일치(value mismatch), CUDA 에러 등으로 분류하는 체계를 수립하고, 모델별 에러 분포를 분석하였다. 컴파일 에러율이 모델의 Triton 코드 생성 능력을 나타내는 유효한 프록시(proxy)임을 밝혔으며, 값 불일치가 정확도 향상의 핵심 병목임을 확인하였다.

본 논문의 구성은 다음과 같다. 제 2절에서 관련 연구를 소개하고, 제 3절에서 시스템 설계를, 제 4절에서 실험 설정을 기술한다. 제 5절에서 실험 결과를 제시하고, 제 6절에서 논의를 수행한다. 마지막으로 제 7절에서 결론을 제시한다.

2 관련 연구

2.1 LLM 기반 코드 생성

대규모 언어 모델의 코드 생성 능력은 Codex [5]의 등장 이후 급속히 발전하였다. Codex는 HumanEval에서 28.8%의 pass@1을 달성하였으며, 이후 CodeGen [37], StarCoder [31] 등의 오픈소스 모델이 다양한 프로그래밍 언어에 대한 코드 생성 능력을 확장하였다. GPT-4 [38]는 HumanEval에서 67%의 pass@1을 달성하며 질적 도약을 이루었고, DeepSeek-Coder [16], CodeLlama [43], Qwen-Coder [21] 등의 오픈소스 모델도 상용 모델에 근접하는 성능을 보이고 있다. HumanEval [5], MBPP [3], SWE-bench [23] 등의 벤치마크가 코드 생성 능력 평가에 활용되고 있으나, 이들은 주로 범용 소프트웨어 개발에 초점을 맞추고 있어 GPU 커널과 같은 도메인 특화 코드 생성 평가에는 한계가 있다.

2.2 LLM 기반 DSL 코드 생성

LLM의 코드 생성 연구는 Python, Java 등 범용 언어에 집중되어 왔으나, 도메인 특화 언어(DSL)에 대한 코드 생성은 근본적으로 더 어려운 문제이다. DSL은 학습 데이터가 범용 언어 대비 극히 제한적이며, 도메인 고유의 의미론과 제약 조건에 대한 이해를 요구하기 때문이다. Thakur 등 [49]은 하드웨어 기술 언어인 Verilog를 대상으로 VeriGen 모델을 제안하여 LLM 기반 DSL 코드 생성의 가능성을 보였으며, Joel 등 [24]은 저자원 및 도메인 특화 언어에 대한 LLM 기반 코드 생성 연구를 체계적으로 정리하였다. GPU 프로그래밍 분야에서 Triton은 PyTorch 에코시스템과의 통합으로 빠르게 채택이 확산되고 있는 대표적인 DSL이며, FlashAttention [11, 12] 등 고성능 커널 구현에 핵심적으로 활용되고 있어 LLM 기반 자동 생성의 중요한 대상이 되고 있다.

2.3 GPU 커널 최적화 및 자동 생성

전통적인 GPU 커널 최적화는 컴파일러 및 탐색 기반 접근에 의존해 왔다. TVM [6]은 텐서 연산을 위한 컴파일러 프레임워크로 스케줄 기반의 최적화 탐색 공간을 정의하며, Ansor [60]는 자동 스케줄 탐색을 통해 수동 튜닝 없이 효율적 커널을 생성한다. Halide [41]는 알고리즘과 스케줄의 분리를 통해 이미지 처리 파이프라인의 최적화를 용이하게 한다.

한편, GPU 커널 개발의 생산성을 높이기 위한 도메인 특화 언어도 발전하고 있다. OpenAI Triton [50]은 블록 수준의 프로그래밍 모델을 채택하여 Python 수준의 사용성과 GPU 수준의 성능을 동시에 추구하며, ThunderKittens [46]는 하드웨어 추상화를 통해 간결하면서도 고성능인 AI 커널 작성을 지원한다. Mirage [54]는 다층 슈퍼옵티마이저로서 텐서 프로그램의 등가성 검증을 통한 자동 최적화를 수행한다. 이러한 접근들은 LLM을 사용하지 않고 컴파일러, 탐색, 또는 형식 검증 기법에 기반하며, 커널 코드 자체의 자동 생성보다는 기존 코드의 최적화에 초점을 맞추고 있다.

2.4 LLM 기반 GPU 커널 생성: 벤치마크와 학습 기반 접근

LLM을 활용한 GPU 커널 생성 연구는 최근 급속히 확대되고 있으며, 이 분야의 연구 동향은 여러 서베이를 통해 체계적으로 정리되고 있다 [15, 56, 58]. 벤치마크 측면에서 KernelBench [40]는 PyTorch 레퍼런스로부터 등가의 최적화된 GPU 커널 생성을 평가하는 250개 문제의 3단계 벤치마크를 수립하여 사실상의 표준이 되었으며, TritonBench [30]는 184개의 실제 GitHub Triton 연산자를 수집하여 Triton에

특화된 벤치마크를 제공한다. 최근 TritonGym [1]은 컴파일러 · 검증기 · 프로파일러 등의 도구 접근을 function-call API로 표준화하여, 모델의 내재 능력과 에이전트 워크플로우 설계를 분리하여 평가할 수 있는 환경을 제안하였다.

강화 학습(RL) 기반 접근은 도메인 특화 모델을 학습시켜 커널 생성 능력을 높인다. Triton 영역에서 AutoTriton [32]은 SFT와 GRPO를 결합하여 8B 규모 모델로 상용 모델에 필적하는 성능을 달성하였고, TritonRL [53]은 세분화된 검증 가능 보상으로 reward hacking을 억제하며, DRTriton [17]은 대규모 합성 데이터로 RL의 데이터 부족 문제를 완화한다. CUDA 영역에서는 CUDA-L1 [33]이 대조적 강화학습(contrastive RL)을, Kevin [4]이 커널 최적화의 반복적 특성을 반영한 멀티턴 RL을 제안하였으며, 나아가 Tehrani 등 [48]은 GPT-5와 같은 프론티어 모델을 RL로 파인튜닝하여 단일 시도 정확도를 크게 끌어올렸다. 특히 Dr. Kernel [36]은 커널 RL의 두 실패 모드인 *reward hacking* (`@triton.jit`만 부착하고 실제 호출하지 않아 측정을 부풀림)과 *lazy optimization*(사소한 sub-op만 교체하여 병목 융합을 놓침)을 명시적으로 규명하고, 분산 실행 환경 KernelGYM과 자기 포함(self-inclusion) 편향을 제거한 멀티턴 이득 추정기 TR-LOO, 프로파일링 기반 보상/거부 샘플링으로 이를 완화하여 14B 모델로 상용 플래그십과 경쟁하는 성능을 보고한다 – 이는 본 연구가 관측한 "검증 강도와 서빙 · 프레임워크 예산이 측정을 좌우한다"는 문제의식과 직접 상통한다. 가장 최근에는 대규모 에이전틱 RL을 결합한 CUDA Agent [10]가 KernelBench 전 레벨에서 `torch.compile`을 능가하며 상용 프론티어 모델을 상회하였고, MusaCoder [7]는 데이터 합성 · 거부 미세조정 · 실행 피드백 RL을 결합한 full-stack 학습으로 신흥 가속기(Moore Threads MUSA)에서도 native 커널 생성을 실증하였다. 이러한 학습 기반 접근은 강력하지만, 모델 훈련을 위한 데이터 큐레이션과 상당한 연산 자원을 요구한다는 공통적 진입 장벽을 가진다.

2.5 에이전트 기반 커널 최적화 프레임워크

추가 훈련 없이(training-free) 기성 LLM을 반복적 에이전트 루프로 활용하는 연구 흐름이 빠르게 부상하고 있다. AMD의 GEAK [52]은 Generator, Reflector, Evaluator, Optimizer의 네 모듈로 구성된 에이전트로 추론 시간 컴퓨트 스케일링을 통해 고성능 Triton 커널을 생성하며, CudaForge [59]는 Coder와 Judge 두 에이전트가 협력하고 Nsight Compute 하드웨어 프로파일링 메트릭을 직접 통합한다. EGG [18]는 전문가 워크플로우를 모사하여 커널 생성을 알고리즘 구조 설계와 하드웨어 특화 튜닝(병렬 매핑 · 타일링 · 메모리 최적화)의 단계로 분해하고 단계 인식 다중 에이전트로 안정적 최적화 궤적을 확보하며, KernelBand [42]은 커널 최적화를 확장하는 행동 공간을 갖는 다중 무장 밴딧(multi-armed bandit)으로 정식화하여, 하드웨어 프로파일 기반 가지치기와 런타임 행동 클러스터링으로 최적화 전략 선택을 원리적으로 유도한다. STARK [13]는 커널 최적화를 Plan/Code/Debug 역할로 분해하고, PRAGMA [28]는 프로파일링 정보를 명시적 추론 단계로 변환하는 다중 에이전트를 제안한다. AccelOpt [57]은 경험 메모리를 활용한 자기 개선으로 신규 가속기 커널을 최적화하며, Meta의 KernelEvolve [35]은 트리 탐색 기반 자기 개선 상태 머신을 NVIDIA · AMD · 자체 가속기 등 이종 플랫폼에 확장하여 프로덕션 규모의 적용 가능성을 입증하였다. AdaExplore [14]는 반복되는 실패를 재사용 가능한 검증 규칙으로 distill하는 failure-driven adaptation과 다양성 보존 탐색(diversity-preserving search)을 결합하여, 단순 반복 개선의 한계를 극복한다. 이 흐름은 본 연구와 동형의 생성 · 진단 · 평가 분리 구조를 공유하는데, KForge [45]는 생성 에이전트와 성능 분석 에이전트가 정확성 단계(functional pass)와 최적화 단계(optimization pass)를 교대하며 NVIDIA · Intel 이종 가속기를 동시에 겨냥하고, Kernel Foundry [20]는 구조화된 진단 피드백 기반의 multi-island 진화 탐색과 중앙 경험 라이브러리를 결합한다. AutoMegaKernel [22]은 정적 스케줄-IR 검증기로 잘못된 제안을 실행 전 거부하여 정확성을 *구조의 속성*으로 끌어올리며, AgentCompile [34]은 LLM 출력을 보조 탐색 메타데이터로만 사용하고 정확성 결정을 컴파일러로 분리하여 silent miscompilation 위험을 차단한다. 개별 커널을 넘어 전체 모델 그래프로 범위를 확장한 LEGO [55]는 `nn.Module` 경계를 구조적 사전지식으로 삼아 top-down 분해(정확성)와 bottom-up 투기적 융합(성능)을 결합한 재귀적 AND-OR FusionTree로 end-to-end 텐서 그래프를 최적화하며, 이는 본 연구에서 Level 2/3의 복합 융합 문제가 겪는 컨텍스트 과부하와 예산 초과를 완화하는 구조적 처방을 시사한다. 한편 raw CUDA 대신 컴팩트 DSL을 노출하여 LLM의 추론 부담을 줄이는 접근으로 NVIDIA의 μCUTLASS [19](CUTLASS DSL + Speed-of-Light guidance), CuTeGen [44](CuTe 기반 progressive refinement와 delayed profiling), QiMeng-Kernel [61](Macro-Thinking Micro-Coding)이 제안되었고, SwizzlePerf [51]는 프로파일링 로그와 아키텍처 스펙을 명시적 하드웨어 인식 컨텍스트로 구성한다.

2.6 정확성 검증과 본 연구의 위치

LLM이 생성한 커널은 컴파일은 되지만 의미적으로 틀릴 수 있어 정확성 검증이 핵심 과제로 부각되고 있다. Kwon 등 [26]은 컴파일러와 런타임이 협력하는 검증 사슬(Chain of Verification)을 인프라 수준에서 제안하였고, COLT [47]는 여러 소형 LLM이 공유 MCTS 위에서 협력하며 모델 회귀 시 큰 모델로 escalate하는 다중 모델 협력 기법을 제시한다. 코드 후보 선택 측면에서 Crupi 등 [9]은 소형 LM을 judge로 학습시켜 저비용으로 다수 후보 중 정답을 선별하는 방법을 보였다.

본 연구는 이러한 기존 연구들과 여러 측면에서 차별화된다. 첫째, 본 연구의 파이프라인은 GEAK [52]과 유사한 생성–평가–리플렉션–최적화 구조를 공유하나, GEAK이 AMD GPU를 대상으로 한 것과 달리 RTX PRO Blackwell(sm_120)을 타깃으로 하며, TritonRL이나 AutoTriton과 같이 RL 기반 모델 훈련을 요구하지 않고 상용 API 모델을 추가 훈련 없이 그대로 활용하여 진입 장벽을 낮춘다. 둘째, 단일 모델 평가에 집중하는 기존 연구들과 달리, 이종 모델을 결합하는 다중 모델 캐스케이딩 전략을 제안하고 그 효과를 실증한다. 이는 COLT [47]의 다중 모델 협력이나 SLM judge [9]와 문제의식을 공유하지만, 별도 학습 없이 이종 상용 모델의 확률적 다양성을 활용한다는 점에서 구별된다. 셋째, 모델별 에러 분류 체계를 통한 실패 모드 분석을 제시하며, 특히

AdaExplore [14]가 탐색 다양성의 중요성을 보인 것과 맥을 같이하여, 확률적 다양성(stochastic diversity)이 반복 심화(iteration depth)보다 효과적이라는 발견을 정량적으로 입증한다.

2.7 추론 모델과 코드 생성

최근 o1 [39], gpt-5.2-pro 등의 추론(reasoning) 모델이 등장하여 수학적 추론과 복잡한 논리 문제에서 우수한 성능을 보이고 있다. 그러나 코드 생성 과제에서는 추가적인 추론 단계가 오히려 실행 가능한 코드 생성을 방해할 수 있다는 관찰이 보고되고 있다. 본 연구는 이러한 관찰을 GPU 커널 생성이라는 구체적 도메인에서 실증적으로 검증하며, gpt-5.2-pro가 Level 2 및 Level 3에서 0%의 정확도를 보이는 결과를 통해 추론 모델의 코드 생성 한계를 정량적으로 제시한다.

3 시스템 설계

본 절에서는 LLM 기반 Triton GPU 커널 자동 생성 시스템의 전체 구조와 각 구성 요소의 설계를 기술한다.

3.1 파이프라인 개요

제안하는 시스템은 PyTorch 레퍼런스 코드로부터 최적화된 Triton GPU 커널을 자동 생성하고 평가하는 엔드투엔드(end-to-end) 파이프라인으로 구성된다. Figure 1에 전체 파이프라인의 구조를 도시하였다. 파이프라인은 다음의 단계로 구성된다.

Figure 2에 제안 시스템의 전체 모듈 구조를 도시하였다.

1. **문제 로딩**: KernelBench [40]로부터 PyTorch 레퍼런스 코드를 포함한 문제를 로딩한다.
2. **프롬프트 구성**: `PromptBuilder`가 대상 GPU의 하드웨어 사양, Triton 프로그래밍 패턴, 시스템 프롬프트를 결합하여 LLM에 전달할 프롬프트를 구성한다.
3. **코드 생성**: LLM은 OpenAI API 또는 로컬 vLLM 서버를 통해 Triton 커널 코드를 생성한다.
4. **응답 파싱**: `ResponseParser`가 생성된 응답에서 코드 블록을 추출하고 클래스명 정규화 및 산문(prose) 제거 등의 후처리를 수행한다.
5. **평가**: `Evaluator`에 의해 정확성과 성능이 평가된다. 정확성 검증은 5개의 무작위 입력에 대해 레퍼런스 출력과의 절대 오차가 10^{-2} 이내인지 확인하며, 성능 벤치마크는 10회 시행의 중앙값으로 측정한다.
6. **반복적 개선**: 평가에 실패한 경우, `Reflector`가 에러를 분류하고 피드백을 생성하여 LLM에 재전달함으로써 반복적 개선(iterative refinement)을 수행한다. 이 과정은 최대 N회까지 반복된다.

3.2 프롬프트 설계

프롬프트 설계는 LLM이 올바른 Triton 커널을 생성하는 데 핵심적인 역할을 한다. 본 시스템의 프롬프트는 세 가지 구성 요소로 이루어진다.

시스템 프롬프트 GPU의 물리적 리소스 제약 조건을 명시적으로 포함한다. 특히, 공유 메모리의 크기가 약 100KB(101,376 bytes)로 제한되며, `BLOCK_SIZE`를 적절히 설정해야 한다는 가이드라인을 포함한다. 예를 들어, 행렬 곱셈의 경우 `BLOCK_SIZE=64` 이하, element-wise 연산의 경우 `BLOCK_SIZE=1024`를 권장한다.

사용자 프롬프트 PyTorch 레퍼런스 코드 전체와 대상 GPU의 사양(이름, 아키텍처, VRAM 용량, 메모리 대역폭, 이론적 TFLOPS)을 포함한다.

개선 프롬프트 반복적 개선 단계에서 사용되며, 이전에 생성된 코드, 발생한 에러 메시지, 현재 반복 횟수를 포함한다. 이를 통해 LLM은 이전 시도의 실패 원인을 인지하고 수정된 코드를 생성할 수 있다.

출력 형식은 단일 Python 코드 블록으로 제한하며, `torch.nn.Module`을 상속하는 `Model` 클래스와 `@triton.jit` 데코레이터가 적용된 커널 함수를 포함하도록 요구한다.

3.3 CUDA 에러 격리

Triton 커널 평가 과정에서 발견된 중요한 기술적 문제는 CUDA 에러의 전파(cascading) 현상이다. 하나의 잘못된 커널이 실행 중 CUDA 컨텍스트를 오염시키면, 동일 프로세스 내에서 실행되는 모든 후속 커널 평가가 실패하게 된다. 이는 CUDA 런타임이 프로세스 수준에서 상태를 관리하기 때문에 발생하는 근본적인 제약이다.

이 문제를 해결하기 위해, 각 커널 평가를 독립적인 서브프로세스(subprocess)에서 실행하는 격리 메커니즘을 설계하였다. 서브프로세스는 커널을 로딩하고 정확성 및 성능 평가를 수행한 후, 결과를 JSON 형식으로 표준 출력(stdout)에 기록한다. 부모 프로세스는 서브프로세스의 출력을 파싱하여 평가 결과를 수집한다. 이 설계는 250개 문제를 순차적으로 평가하는 대규모 실험에서 필수적이며, 이를 도입하지 않을 경우 단일 실패가 전체 실험 결과를 무효화할 수 있다.

3.4 다중 모델 캐스케이딩 전략

단일 모델의 한계를 극복하기 위해, 복수의 LLM을 순차적으로 활용하는 캐스케이딩(cascading) 전략을 설계하였다.

- **Phase 1**: 전체 문제 집합에 대해 gpt-4o를 실행한다.
- **Phase 2**: Phase 1에서 실패한 문제에 대해 gpt-5.2를 실행한다.
- **Phase 3**: Phase 1과 2 모두에서 실패한 문제에 대해 gpt-5.2-pro를 실행한다.

최종 결과는 각 단계에서 해결된 문제 집합의 합집합(union)으로 산출된다. 이 접근법은 각 모델이 서로 다른 유형의 문제에 강점을 보인다는 관찰에 기반한다. Figure 3에 캐스케이딩 전략의 구조와 각 단계별 성능을 도시하였다.

3.5 다중 API 지원

본 시스템은 다양한 LLM 제공자의 API를 통합적으로 지원하기 위해 자동 API 감지 및 폴백(fallback) 메커니즘을 구현하였다.

- **Chat Completions API**: gpt-4o-mini, gpt-4o, gpt-5.2 등 일반적인 채팅 모델에 사용된다.
- **Responses API**: gpt-5.2-pro와 같은 추론 모델에 사용된다. `temperature` 파라미터를 지원하지 않으며,

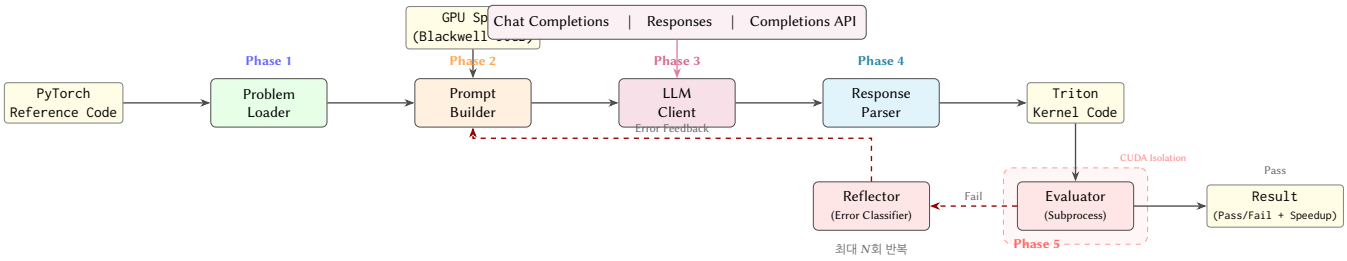


Figure 1. LLM 기반 Triton GPU 커널 자동 생성 시스템의 전체 파이프라인. PyTorch 레퍼런스 코드로부터 프롬프트를 구성하고, LLM이 Triton 커널을 생성하며, 평가 실패 시 에러 피드백을 통해 반복적으로 개선한다. 각 커널 평가는 CUDA 에러 격리를 위해 독립적인 서브프로세스에서 수행된다.

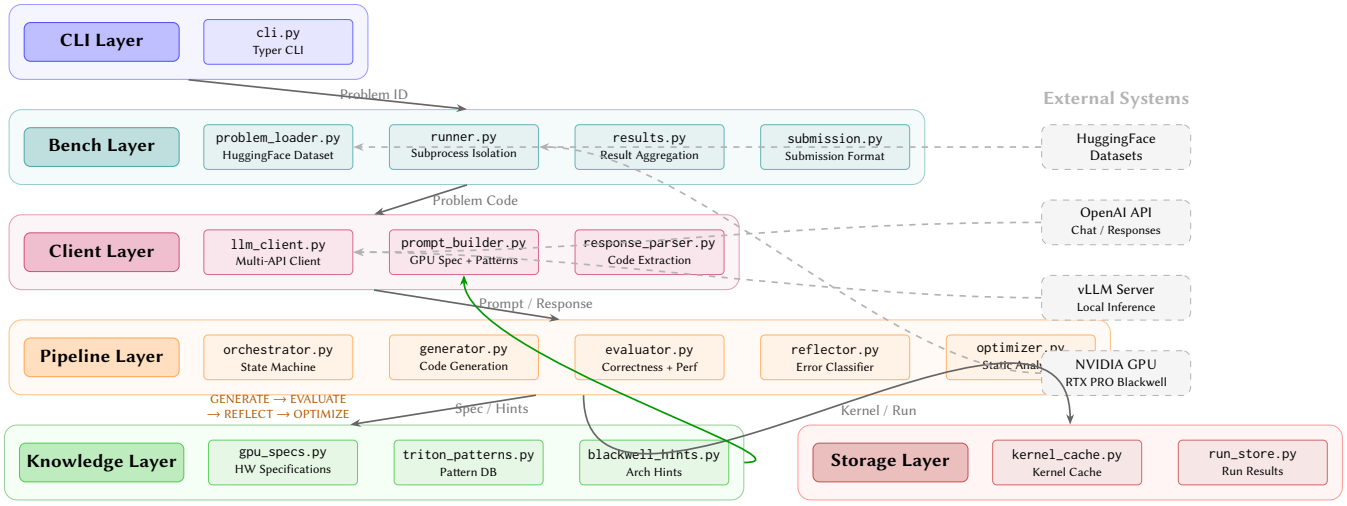


Figure 2. 제안 시스템의 전체 모듈 구조. CLI 레이어가 사용자 입력을 받아 Bench 레이어에서 KernelBench 문제를 로딩하고, Client 레이어가 프롬프트 구성 및 LLM 응답 파싱을 담당한다. Pipeline 레이어의 상태 머신(GENERATE → EVALUATE → REFLECT → OPTIMIZE)이 반복적 생성-평가 루프를 조율하며, Knowledge 레이어의 GPU 사양 및 Triton 패턴이 프롬프트 품질을 향상시킨다. 생성된 커널과 실험 결과는 Storage 레이어에 저장된다.

시스템은 모델 식별자를 기반으로 이를 자동으로 감지한다.

- **Completions API**: 로컬 vLLM 서버에서 제공하는 오픈소스 모델(예: Qwen 2.5 Coder)에 사용된다.

이러한 다중 API 지원은 상용 모델과 오픈소스 모델을 단일 프레임워크 내에서 일관되게 비교 평가할 수 있게 한다.

3.6 구현 기여

본 연구의 시스템은 약 2,500 라인의 Python 코드로 구현되었으며, 재현 가능한 오픈소스 프레임워크로 공개하였다. 구현상의 주요 기술적 기여는 다음과 같다.

***GPU* 인식 프롬프트 엔지니어링** 프롬프트에 대상 GPU의 공유 메모리 크기(101,376 bytes), VRAM 용량, 메모리 대역폭, 이론적 TFLOPS 등 하드웨어 사양을 명시적으로 포함한다. 또한 Triton 프로그래밍 패턴 데이터베이스(`triton_patterns.py`)와 Blackwell 아키텍처 최적화 힌트(`blackwell_hints.py`)를 프롬프트에 주입하여, LLM이 하드웨어 특성에 맞는 커널을 생성하도록 유도한다.

자동 *API* 감지 및 폴백 `LLMClient`는 모델 식별자를 기반으로 Chat Completions API, Responses API, Completions API 중 적절한 API를 자동으로 선택한다. 추론 모델(gpt-5.2-pro, o1, o3)은 `temperature` 파라미터를 지원하지 않으므로, 이를 자동으로 감지하여 Responses API로 전환한다. 비채팅 모델은 Completions API로 폴백하며, 한번 감지된 모델 유형은 캐싱하여 이후 호출에서 재시도를 방지한다.

응답 후처리 파이프라인 `ResponseParser`는 LLM 응답에서 Markdown 코드 블록을 추출하고, 클래스명을 `Model`로 정규화하며, Triton 코드 외의 산문(prose)을 제거한다. 이 후처리는 다양한 LLM의 출력 형식 차이를 흡수하여 일관된 평가를 가능하게 한다.

Table 1. 실험에 사용된 LLM 모델.

모델	유형	API	파라미터	반복 횟수
gpt-4o-mini	Chat	Chat Completions	비공개	5
gpt-4o	Chat	Chat Completions	비공개	5
gpt-5.2	Chat	Chat Completions	비공개	5
gpt-5.2-pro	Reasoning	Responses	비공개	3
Qwen 2.5 Coder	Chat	Completions (vLLM)	32B	5
gpt-5.4-nano	Chat	Chat Completions	비공개	3
gpt-5.5	Chat	Chat Completions	비공개	3
claude-opus-4-7	Chat	Chat Completions	비공개	3

상태 머신 기반 오케스트레이션 `Orchestrator`는 GENERATE → EVALUATE → REFLECT → OPTIMIZE의 4단계 상태 머신으로 구현되며, 각 단계의 전이 조건과 최대 반복 횟수를 설정 가능하게 설계하였다. `Reflector`는 에러 메시지를 분석하여 컴파일 에러, 값 불일치, CUDA 에러, 타임아웃 등으로 자동 분류하고, 에러 유형에 맞는 피드백 프롬프트를 생성한다.

4 실험 설정

4.1 벤치마크: KernelBench

본 연구의 평가에는 KernelBench [40]를 사용한다. KernelBench는 PyTorch로 작성된 레퍼런스 코드를 등가의 Triton GPU 커널로 변환하는 능력을 측정하기 위해 설계된 벤치마크로, HuggingFace datasets를 통해 제공된다.

- **Level 1** (100 문제): 단일 연산 수준의 문제로, element-wise 연산, reduction, 행렬 곱셈 등 기본적인 GPU 연산을 포함한다.
- **Level 2** (100 문제): 복수 연산의 조합으로 구성된 문제로, attention 메커니즘, convolution 연산 등 중간 수준의 복잡도를 갖는다.
- **Level 3** (50 문제): 전체 모델 또는 모델 블록 수준의 문제로, ResNet 블록, Transformer 레이어 등을 포함하며 가장 높은 난이도를 갖는다.

4.2 평가 모델

Table 1은 본 실험에서 평가한 LLM의 목록이다.

OpenAI의 상용 모델은 파라미터 수가 공개되지 않으며, API를 통해서만 접근 가능하다. Qwen 2.5 Coder 32B [21]는 로컬 vLLM 서버에서 실행하였다. gpt-5.2-pro는 추론(reasoning) 특화 모델로, API 호출 비용과 응답 지연을 고려하여 반복 횟수를 3회로 설정하였다. 확장 실험에서 추가된 gpt-5.4-nano, gpt-5.5, claude-opus-4-7은 Mindlogic AI Gateway[1]를 통합 API 엔드포인트로 사용하여 평가하였으며, 반복 횟수는 3회로 설정하였다.

4.3 평가 메트릭

KernelBench에서 정의한 세 가지 메트릭을 사용한다 [40].

- **Fast-0 (정답률)**: 생성된 커널이 정확한 결과를 출력하는 비율. 5개의 무작위 입력에 대해 레퍼런스 출력과의 절대 오차(absolute tolerance)가 10^{-2} 이내이면 정답으로 판정한다.

[1] https://api.mindlogic.ai

Table 2. 모델별 KernelBench 정확도 비교(Fast-0). †gpt-4o 실패 문제에 대해서만 실행(cascading strategy). ‡Level 1만 평가(확장 실험). §Level 1 P1–20 부분집합을 KernelAgent(best-of-4)로 평가한 RL 특화 오픈 모델.

Model	L1 (100)	L2 (100)	L3 (50)	Total (250)
Qwen 2.5 Coder 32B	0 (0%)*	—	—	0*
gpt-4o-mini	15 (15%)	1 (1%)	0 (0%)	16 (6.4%)
gpt-4o	19 (19%)	6 (6%)	0 (0%)	25 (10%)
gpt-5.4-nano‡	35 (35%)	—	—	—
gpt-5.2†	23/81 (28.4%)	9/94 (9.6%)	4/50 (8%)	36/225 (16%)
gpt-5.5‡	37 (37%)	—	—	—
gpt-5.2-pro†	13/81 (16%)	0/94 (0%)	0/50 (0%)	13/225 (5.8%)
gemini-3.1-pro-preview‡	43 (43%)	—	—	—
claude-opus-4-7‡	**46 (46%)**	—	—	—
Dr.Kernel-14B§ (RL, open)	13/20 (65%)	—	—	—

*파이프라인 초기 버전의 응답 파서 결함으로 모든 출력이 구문 오류로 실패. 모델 자체의 코드 생성 능력을 반영하지 않음.

- **Fast-1**: 정답인 동시에 speedup ≥ 1.0×인 비율.
- **Fast-2**: 정답인 동시에 speedup ≥ 2.0×인 비율.

Speedup은 speedup = $t_{\text{ref}}/t_{\text{gen}}$으로 계산하며, 각 시간은 10회 시행의 중앙값으로 측정한다.

4.4 하드웨어 환경

주 실험은 NVIDIA RTX PRO 6000 Blackwell(96GB VRAM, sm_120)에서 수행하였으며, 하드웨어 의존성 분석을 위해 NVIDIA H100(80GB, sm_90)과 RTX A6000(48GB, Ampere)에서 교차 평가하였다(§5.7). 소프트웨어 환경은 Triton 3.5.1(H100)/3.6(Black PyTorch 2.x, Python 3.12, CUDA 12.x로 구성하였다.

4.5 실험 설계

실험 *1:* 모델 간 비교 5개 모델에 대해 KernelBench의 3개 레벨 전체(250개 문제)를 평가한다. 다중 모델 캐스케이딩 전략의 효과를 측정하기 위해 gpt-4o, gpt-5.2, gpt-5.2-pro를 순차적으로 적용한 결과를 분석한다. 확장 실험으로 gpt-5.4-nano, gpt-5.5, claude-opus-4-7을 Level 1(100문제)에 대해 추가 평가하고, 5개 모델 합집합의 성능을 분석한다.

실험 *2:* 반복 횟수 *ablation*. gpt-4o 모델과 Level 1 문제를 대상으로 반복 횟수를 {1, 3, 5, 10}으로 변화시키며 실험한다.

실험 *3:* 에러 분석 전체 실패 사례의 에러 유형을 분류하고 레벨별, 모델별 에러 분포를 분석한다.

5 실험 결과

5.1 모델 비교 결과

Table 2은 각 모델의 난이도별 정확도를 보여준다.

확장 실험을 포함한 전체 결과에서, **claude-opus-4-7이 단일 모델 최고 성능(46/100, 46%)**을 달성하였다. 이는 기존 최고였던 gpt-5.2의 캐스케이딩 기준 28.4%를 크게 상회하며, 이전 3-모델 캐스케이딩 전략(43/100)조차 초과하는 결과이다. gemini-3.1-pro-preview(43/100, 43%), gpt-5.5(37/100, 37%), gpt-5.4-nano(35/100, 35%)도 기존 gpt-4o(19%)를 크게

Table 3. 다중 모델 캐스케이딩 전략의 누적 성능.

Configuration	L1 (/100)	L2 (/100)	L3 (/50)	Total (/250)
gpt-4o-mini	15 (15%)	1 (1%)	0 (0%)	16 (6.4%)
gpt-4o	19 (19%)	6 (6%)	0 (0%)	25 (10%)
gpt-4o + gpt-5.2	42 (42%)	15 (15%)	4 (8%)	61 (24.4%)
+ gpt-5.2-pro	43 (43%)	15 (15%)	4 (8%)	62 (24.8%)
claude-opus-4-7 (단일)	46 (46%)	—	—	—
opus + gpt-5.4-nano	48 (48%)	—	—	—
전체 5모델 union	**51 (51%)**	—	—	—

상회하여, 최신 세대 모델의 Triton 커널 생성 능력이 전반적으로 향상되었음을 확인하였다.

특히 주목할 현상은 gpt-5.5와 claude-opus-4-7이 처음 44문제(P1–P44)에서 **정확히 동일한 37개 문제**를 해결했다는 점이다. P4만을 유일한 공통 실패로 공유하며 나머지 43문제에서 완벽히 일치하는 성공/실패 패턴을 보였다. 그러나 P45 이후 후반부에서 두 모델은 극적으로 분기하였다: gpt-5.5는 추가 0개, claude-opus-4-7은 추가 9개(P88–P100)를 해결하였다. 이러한 **후반부 회복 능력(late recovery)**이 claude-opus-4-7을 단일 모델 최고 성능으로 이끈 핵심 차별화 요인이다. 흥미롭게도 3사 플래그십인 gemini-3.1-pro-preview 역시 후반부(P88–P100)에서 6문제를 회복하며 43/100을 기록하여, late recovery가 특정 모델 고유의 능력이 아니라 *최신 플래그십 세대의 공통 특성*으로 확산되고 있음을 시사한다.

기존 모델 중 gpt-5.2는 시도한 문제 대비 높은 성공률을 달성하였으며, Level 3에서도 유일하게 4문제를 해결하여 복잡한 커널에 대한 생성 능력이 우수함을 보여준다. 반면, gpt-5.2-pro는 추론 특화 모델(reasoning model)임에도 불구하고 코드 생성 태스크에서 기대 이하의 성능을 보였다. Level 1에서 16%에 그쳤으며, Level 2와 Level 3에서는 단 하나의 문제도 해결하지 못하였다. 이는 해당 모델의 긴 추론 시간으로 인한 타임아웃이 주요 원인이다(Section 5.4 참조).

Level 3은 gpt-4o와 gpt-4o-mini 모두 50문제 중 단 하나도 해결하지 못하여, 심각한 난이도 절벽(difficulty cliff)이 존재함을 확인하였다. Qwen 2.5 Coder 32B의 경우, 파이프라인 초기 버전의 응답 파서 결함으로 인해 모든 생성 결과가 구문 오류로 실패하여 공정한 비교가 불가능하였다.

5.2 다중 모델 캐스케이딩 결과

서로 다른 LLM이 질적으로 다른 문제를 해결할 수 있다는 가설 하에, 다중 모델 캐스케이딩 전략을 실험하였다. Table 3에 결과를 정리하였다.

기존 캐스케이딩 전략의 효과는 현저하다. gpt-4o 단독 25문제에서 gpt-5.2를 추가함으로써 61문제로 증가하여, 36개의 추가 문제를 해결하였다(**2.44배 향상**). 이는 gpt-5.2가 gpt-4o와 질적으로 다른 문제 해결 능력을 보유함을 의미한다. 구체적으로, Level 1에서 23개, Level 2에서 9개, Level 3에서 4개의 추가 문제를 해결하였다.

반면, gpt-5.2-pro의 추가 기여는 미미하였다. 전체 225문제를 시도하여 단 1개의 추가 문제(Level 1의 Problem 88, 7.60× speedup)만을 해결하였다. 이는 높은 API 비용과 긴

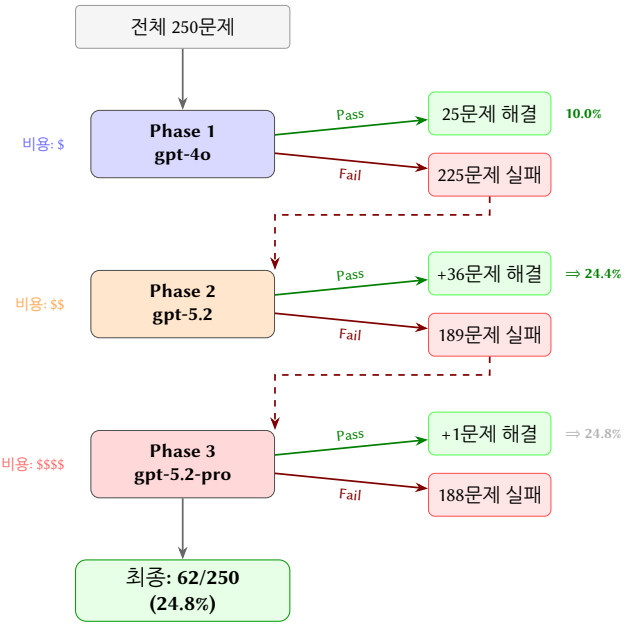


Figure 3. 다중 모델 캐스케이딩 전략. 각 단계에서 이전 단계의 실패 문제만을 대상으로 상위 모델을 적용한다. gpt-4o→gpt-5.2 단계에서 2.44배의 성능 향상이 달성되며, gpt-5.2-pro 추가의 비용 대비 효과는 미미하다.

Table 4. 반복 횟수에 따른 성능 및 에러 유형 변화 (gpt-4o, Level 1).

Iter.	Solved (/100)	Compile Err.%	Val. Mismatch%
1	19 (19%)	59.3%	24.7%
3	21 (21%)	51.9%	39.2%
5	19 (19%)	45.7%	37.0%
10	22 (22%)	32.1%	53.8%
Union	**28 (28%)**	—	—

응답 시간을 고려할 때, 캐스케이딩 전략에서의 비용 대비 효과가 극히 낮음을 시사한다.

확장 실험에서는 최신 모델을 포함한 합집합 분석을 수행하였다. claude-opus-4-7 단일 모델(46/100)이 이전 3-모델 캐스케이딩(43/100)을 초과한 것이 가장 주목할 결과이다. claude-opus-4-7과 gpt-5.4-nano의 2-모델 합집합은 48/100으로, gpt-5.4-nano가 opus가 해결하지 못한 2개의 추가 문제를 해결하였다. 5개 모델 전체의 합집합은 **51/100(51%)**으로, 최선의 단일 모델(46/100) 대비 10.9% 향상을 달성하여, 확률적 다양성 원리가 최신 세대 모델에서도 유효함을 확인하였다.

5.3 반복 횟수 Ablation 연구

반복적 개선 전략에서 반복 횟수가 성능에 미치는 영향을 분석하기 위해, gpt-4o 모델로 Level 1 문제에 대한 ablation 실험을 수행하였다. Table 4에 결과를 제시한다.

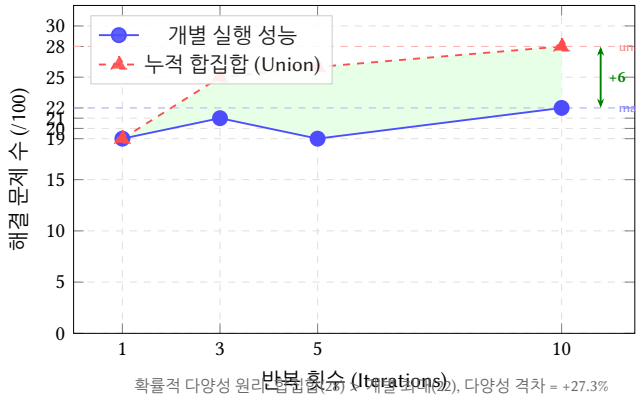


Figure 4. 반복 횟수에 따른 성능 변화 (gpt-4o, Level 1). 개별 실행 성능은 반복 횟수에 따라 미미한 변동을 보이나, 누적 합집합은 꾸준히 증가한다. 이는 반복의 깊이보다 생성의 다양성이 더 효과적임을 시사한다.

Table 5. 모델별 주요 실패 유형 요약.

Model	Primary Failure	Secondary	해석
gpt-4o-mini	Compile (71–82%)	Val. mismatch	Triton 문법 미숙
gpt-4o	Compile (38–51%)	Val. mismatch	문법 개선, 의미 정확성 부족
gpt-5.2	Val. mismatch (59–72%)	Compile (7–26%)	문법 능숙, 의미적 병목
gpt-5.2-pro	Timeout (82–100%)	Rate limit	추론 오버헤드
gpt-5.4-nano	Val. mismatch + Other	CUDA	혼합 에러 프로필
gpt-5.5	Compile (71%)	Timeout (25%)	Triton 문법 미숙
claude-opus-4-7	Val. mismatch (65%)	Timeout (28%)	문법 완벽, 의미적 병목

주목할 점은 개별 실행의 최대 성공 수(22개, 10회 반복)와 합집합(28개) 사이의 격차이다. 이는 각 실행이 서로 다른 문제를 확률적으로 해결함을 의미하며, 반복 깊이를 늘리는 것보다 독립적인 다수 시도를 통한 **확률적 다양성(stochastic diversity)**이 더 효과적인 전략임을 시사한다.

에러 유형의 변화 추이도 의미있다. 반복 횟수가 증가함에 따라 컴파일 에러 비율은 59.3%에서 32.1%로 꾸준히 감소하는 반면, 값 불일치 비율은 24.7%에서 53.8%로 증가한다. 이는 반복적 개선이 Triton 문법 오류를 효과적으로 수정하지만, 의미적 정확성(semantic correctness) 문제는 반복만으로 해결되지 않음을 보여준다.

5.4 에러 분석

생성 실패의 원인을 체계적으로 분류하기 위해, 각 모델의 에러 유형 분포를 분석하였다. Table 5에 요약을, Table 6에 상세 분포를 제시한다.

에러 분석에서 도출되는 핵심 통찰은 다음과 같다.

첫째, **컴파일 에러율은 모델 능력과 역비례한다**: gpt-4o-mini(71–82%) > gpt-4o(38–51%) > gpt-5.2(7–26%). 특히 gpt-5.2의 Level 1 컴파일 에러율이 6.9%에 불과한 것은 Triton 문법을 거의 완벽하게 습득하였음을 의미한다.

둘째, 컴파일 에러가 감소하면 **값 불일치가 지배적 에러로 부상한다**. gpt-5.2에서 값 불일치가 59–72%를 차지하는 것은, 현재 LLM 기반 커널 생성의 핵심 도전이 Triton 문법이 아닌 GPU 연산의 논리적 이해에 있음을 시사한다.

Table 6. 모델 및 난이도별 에러 분포(%). 실패한 문제 중 해당 에러 유형의 비율.

Model	Level	Failed	Compile	Val. Mis.	CUDA	Timeout
gpt-4o-mini	L1	85	71.8%	20.0%	2.4%	2.4%
	L2	99	70.7%	22.2%	4.0%	1.0%
	L3	50	82.0%	6.0%	6.0%	0%
gpt-4o	L1	81	45.7%	37.0%	11.1%	0%
	L2	94	51.1%	41.5%	4.3%	0%
	L3	50	38.0%	56.0%	4.0%	0%
gpt-5.2	L1	58	6.9%	58.6%	1.7%	20.7%
	L2	85	25.9%	62.4%	0%	8.2%
	L3	46	17.4%	71.7%	0%	4.3%
gpt-5.2-pro	L1	68	0%	0%	0%	100%
	L2	94	0%	0%	0%	81.9%
	L3	50	0%	0%	0%	0%*
gpt-5.4-nano	L1	65	–	–	–	–
gpt-5.5	L1	63	71%	3%	0%	25%
claude-opus-4-7	L1	54	0%	65%	0%	28%

*Level 3의 gpt-5.2-pro 실패는 전량 API rate limit(HTTP 429) 에러에 기인.

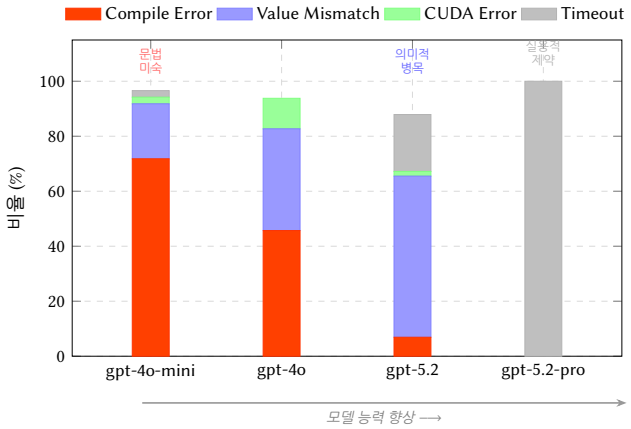


Figure 5. 모델별 에러 유형 분포 (Level 1 기준). 모델 능력이 향상됨에 따라 컴파일 에러에서 값 불일치(value mismatch)로 지배적 에러 유형이 전이되는 패턴이 관찰된다. gpt-5.2-pro의 실패는 전량 타임아웃에 기인한다.

셋째, gpt-5.2-pro의 실패는 모델 능력이 아닌 **실용적 제약에 기인한다**. 추론 모델의 긴 사고 과정(chain-of-thought)이 타임아웃을 유발하여, Level 2에서 81.9%, Level 1에서 100%의 문제가 타임아웃으로 실패하였다.

5.5 속도 개선(Speedup) 분석

정확성 평가를 통과한 커널에 대해 PyTorch 기준 구현 대비 실행 속도 개선을 측정하였다. Table 7에 상위 10개 결과를, Table 8에 Fast-1 및 Fast-2 지표를 제시한다.

가장 주목할 만한 결과는 Level 3 Problem 16에서 gpt-5.2가 달성한 84.03배의 속도 개선이다. 이는 특정 연산 패턴에서 LLM이 생성한 Triton 커널이 PyTorch의 기본 구현을 극적으로 능가할 수 있음을 보여준다.

Fast-1 지표에서 gpt-5.2(69.4%)와 gpt-5.2-pro(76.9%)가 gpt-4o-mini(43.8%)와 gpt-4o(48.0%)를 크게 상회한다. 흥미로운

Table 7. 상위 10개 속도 개선 결과.

Rank	Problem	Level	Model	Speedup
1	P16	3	gpt-5.2	84.03×
2	P9	2	gpt-4o-mini	25.45×
3	P70	2	gpt-4o	19.10×
4	P12	1	gpt-4o	11.86×
5	P88	1	gpt-5.2-pro	7.60×
6	P100	1	gpt-5.2-pro	7.35×
7	P49	2	gpt-4o	6.54×
8	P98	1	gpt-5.2-pro	5.32×
9	P43	3	gpt-5.2	4.26×
10	P99	1	gpt-5.2	4.05×

Table 8. 모델별 Fast-1 (speedup ≥ 1.0×) 및 Fast-2 (speedup ≥ 2.0×) 지표.

Model	Fast-1	Fast-2
gpt-4o-mini	7/16 (43.8%)	1/16 (6.3%)
gpt-4o	12/25 (48.0%)	4/25 (16.0%)
gpt-5.2	25/36 (69.4%)	5/36 (13.9%)
gpt-5.2-pro	10/13 (76.9%)	3/13 (23.1%)
gpt-5.4-nano	21/35 (60.0%)	10/35 (28.6%)
gpt-5.5	22/37 (59.5%)	8/37 (21.6%)
claude-opus-4-7	29/46 (63.0%)	14/46 (30.4%)

점은 gpt-5.2-pro가 해결한 문제 수는 13개로 적지만, Fast-2 비율(23.1%)은 가장 높다는 것이다. 이는 추론 모델이 문제를 해결할 경우 보다 최적화된 커널을 생성하는 경향이 있음을 시사하나, 낮은 전체 성공률을 고려하면 실용적 가치는 제한적이다.

5.6 평가 인프라 결함 진단 및 재측정

확장 분석 과정에서 본 연구는 초기 파이프라인의 평가 정확도를 심각하게 저해하던 인프라 결함을 발견하고 이를 수정한 뒤, 대표 부분집합에 대해 재측정을 수행하였다.

***GPU* 메모리 경합으로 인한 거짓 실패** 오픈소스 모델은 로컬 vLLM 서버로 서빙되는데, 기본 설정(`gpu-memory-utilization` 0.9)에서 vLLM이 GPU 메모리의 약 90%(86GB)를 점유한다. 이 상태에서 평가기(evaluator)가 대형 입력 텐서를 갖는 문제를 채점할 때 CUDA out-of-memory(OOM)가 발생하여, 기능적으로 정확한 커널조차 거짓 실패(false negative)로 판정되었다. 예컨대 Level 1의 ReLU 문제(P19)는 입력 텐서 한 개가 6.4GB에 달하며($4096 \times 393216 \times 4$ bytes), 5회 반복 채점 시 텐서가 해제되지 않고 누적되어 OOM을 유발한다. ReLU와 같이 의미적으로 자명한 문제조차 거짓 실패하던 현상은 이것이 모델 능력이 아닌 평가 인프라의 문제임을 드러내며, 이는 §6.6에서 언급한 오픈소스 모델 평가 불공정의 직접적 원인이다.

Table 9. 인프라 수정 후 Level 1 부분집합(P1–P20, 20문제) 정답률(Fast-0) 재측정. claude-opus-4-7은 부분집합 전체를 정확히 해결하였다.

모델	수정 전	수정 후
Qwen3-Coder-30B-A3B-FP8	0.45	0.75
claude-opus-4-7	0.65	**1.00**

수정 다음 다섯 가지를 적용하였다. (1) 평가기의 정확성 검증 루프에서 시행(trial)마다 GPU 텐서를 명시적으로 해제하고, (2) vLLM 메모리 점유율을 0.45로 낮춰 평가기에 약 52GB를 확보하였다. 아울러 (3) 생성 프롬프트에 **입력 dtype 계약**(레퍼런스 입력의 dtype을 임의로 가정하거나 `float16`으로 캐스팅하지 말 것)을 명시하고, (4) 정확성 검증이 통과해야만 성능 최적화 단계로 진입하는 **게이팅(gating)**을 보장하며, (5) 반복 개선이 일정 횟수 정체되면 누적된 문맥(이전 코드 · 에러 피드백)을 폐기하고 처음부터 재생성하는 **문맥 초기화 재시도(context reset)**를 도입하였다.

재측정 결과 수정된 파이프라인으로 Level 1의 대표 부분집합(20문제, P1–P20)을 재평가한 결과는 Table 9와 같다. Qwen3-Coder-30B는 OOM 수정만으로 정답률이 0.45에서 0.75로 상승하였으며, claude-opus-4-7은 전체 수정을 적용했을 때 **20문제 전부(20/20, 100%)**를 정확하게 해결하였다. 특히 Qwen이 풀지 못하던 3D/4D 텐서 곱셈 및 전치 행렬 곱셈(P10, P11, P16–P18)을 opus가 모두 해결하여, 잔여 실패가 인프라가 아닌 모델 능력의 한계였음을 확인하였다. 다만 본 재측정은 20문제 부분집합에 한정되며, Level 1 전체(100문제) 및 Level 2/3로의 재평가는 향후 과제로 남긴다.

프로파일링 피드백 *Ablation.* NVIDIA Nsight Compute(ncu) 하드웨어 카운터(SM/메모리 처리율, 병목 분류)를 최적화 프롬프트에 주입하는 A/B 실험을 3개 모델(Qwen3-Coder-30B, gpt-5.2, claude-opus-4-7)에 대해 수행하였다. 그러나 ncu 피드백은 어떤 모델에서도 정답률을 유의하게 개선하지 못하였으며(정답률 변화는 ±1–2문제의 표본 노이즈 수준), 속도 개선도 미미하였다. 이는 ncu 프로파일이 *이미 정확한* 커널의 최적화 단계에서만 작동하는 반면, 실제 병목은 정확성(값 불일치)이라는 본 연구의 핵심 진단(§6.1의 에러 전이 가설)과 일치한다. 즉 프로파일링 기반 최적화는 정확성 문제가 선결된 이후에야 의미를 가지며, 이는 *지연 프로파일링*(correctness-first, profiling-later) 설계의 필요성을 실증적으로 뒷받침한다. 이 결과는 자기진화 에이전트에서 *정렬된(aligned) 피드백만이 계획 개선에 유효*하다는 최근의 독립적 분석 [8]과도 일치한다.

5.7 대규모 확장 실험: 전체 레벨 × 다중 하드웨어 × 다중 벤치마크

수정된 파이프라인으로 무료 오픈소스 모델 두 가지(**Qwen3-Coder-30B**, **AutoTriton-8B** [32])를 두 하드웨어(**RTX PRO**

Table 10. 전체 레벨 정답률(Fast-0), best-of-1 · 반복 3. AT=AutoTriton-8B. BW=Blackwell sm_120, H100=sm_90, A6000=Ampere. H100/A6000 AT 열은 2026-06 전체 스위트 재실행 결과이며, best-of-1 특성상 절대값은 실행 간 편차가 있으나 모델 · 하드웨어 간 정성적 순서는 유지된다(‡).

레벨	BW qwen	H100 qwen	BW AT	H100 AT‡	A6000 AT‡
L1 (100)	0.27	0.12†	**0.47**	0.43	0.37
L2 (100)	0.05	0.06	0.10	0.08	**0.12**
L3 (50)	0.04	0.06	**0.06**	0.02	0.04
FlashInfer (8)	0.00	0.00	0.00	**0.25**	0.00

6000 Blackwell, sm_120 / **H100**, sm_90)에서 교차 평가하였다. 빠른 전수 스캔을 위해 best-of-1, 반복 3회로 KernelBench 전체 250문제(L1 100, L2 100, L3 50)와 FlashInfer-Bench[2] 8문제를 측정하였다(Table 10).

세 가지 발견이 두드러진다. 첫째, **무료 8B Triton 특화 모델(AutoTriton)이 30B 범용 모델(Qwen)을 전 레벨에서 일관되게 능가**한다. 별도의 소규모 정밀 실험(L1 1–20, best-of-3, 반복 9)에서는 AutoTriton-8B가 H100에서 0.95, Qwen-30B가 Blackwell에서 0.75의 정답률을 보여, Triton 특화 강화학습의 비용 효율을 뒷받침한다. 둘째, **H100에서 Qwen의 L1 정답률(0.12†)이 비정상적으로 낮은 것은 GPU 메모리 제약** 때문이다. H100(80GB)은 30B 모델 서빙 시 평가기에 충분한 메모리를 남기지 못해 대형 입력 문제(예: ReLU의 6.4GB 입력)가 OOM으로 거짓 실패하며, 8B 모델(AutoTriton)은 메모리 여유로 이 문제가 없다(§5.6). 셋째, **FlashInfer GEMM은 H100 AutoTriton에서만 성공**(0.25)하고 Blackwell과 A6000에서는 0을 기록하여, Triton 버전(3.5.1 vs 3.6)과 컴퓨트 아키텍처에 따른 커널 컴파일 차이를 시사한다. 한편 동일 AutoTriton 모델을 세 번째 하드웨어인 A6000(Ampere, 48GB)에서 재실행한 결과는 KernelBench 전 레벨에서 Blackwell · H100과 동일한 수준의 정답률(L1 0.37, L2 0.12, L3 0.04)을 보여, 메모리가 충분한 조건에서 **정답률이 하드웨어에 거의 독립적**이라는 결론(아래)을 추가로 뒷받침한다. 다만 A6000은 추론 처리량이 H100의 약 1/4–1/5에 그쳐(약 34 대 150 tokens/s) 동일 스위트 수행에 H100 대비 약 3배의 벽시계 시간이 소요되었다.

***TritonBench* 평가** 본 연구가 Triton 커널 생성을 다루므로, Triton 특화 벤치마크인 TritonBench-T[3] [32](166문제)으로도 4조합을 평가하였다(Table 11). 네 조합 모두 Execution Accuracy 0.15–0.19로 낮고 모델/하드웨어 간 격차가 작았다. 처음에는 이 격차를 (1) TritonBench에 optimizer(Adam, SGD) 등 Triton 변환이 본질적으로 부적합한 문제가 포함된 점과, (2) AutoTriton의 보고 성능이 그 학습 프롬프트 · 자체 평가 하니스에 특화된 반면 본 실험은 통일된 변환 프롬프트로 공정 비교한 *분포 외(out-of-distribution)* 조건으로 가설하였다.

[2]NVIDIA Blackwell용 실전 LLM 서빙 커널 벤치마크. 본 연구에서는 dense(GEMM) 8문제를 KernelBench 형식으로 변환해 평가하였다.
[3]각 문제는 PyTorch 함수와 `test_*()` 하니스로 구성되며, 생성된 Triton 구현을 동일 하니스로 실행해 결과 일치(Execution Accuracy)와 실행 가능성(Call Accuracy)을 측정한다.

Table 11. TritonBench-T(166문제) Execution Accuracy(best-of-1).

모델	Blackwell	H100
AutoTriton-8B	0.157	0.151
Qwen-30B	0.163	0.187

프롬프트 형식 대 *test-time scaling Ablation.* 이 가설을 검증하기 위해 프롬프트 형식(PyTorch 레퍼런스를 주는 *convert* 대 AutoTriton의 공식 instruction *official*)과 추론 시간 스케일링(단일 시도 n=1 대 n=3 샘플링+반복 수정)을 교차한 2 × 2 ablation을 수행하였다. 결과는 OOD 가설을 뒤집었다. **프롬프트 형식만 바꾸는 단독 효과는 미미**하여 Execution Accuracy가 0.151에서 0.127로 오히려 하락하기도 한 반면, **test-time scaling을 켜면 Execution Accuracy가 약 2.3배, Call Accuracy가 약 2.5배 상승**하였다. 나아가 스케일링 하에서는 PyTorch 레퍼런스를 제공하는 convert 형식이 프롬프트 정렬보다 오히려 우수하였다. 즉 원본 논문 수치와의 격차는 RL 학습 프롬프트 형식의 분포 차이가 아니라 *단일 시도 평가냐, 반복 수정과 샘플링을 동반하느냐*의 차이로 설명된다. 이는 KernelBench에서 관찰한 결론(§5.7)과 정확히 일치하며, **test-time scaling이 벤치마크를 가리지 않는 일반 원리**임을 재확인한다.

종합하면, **정답률(correctness)은 하드웨어에 거의 독립적**(모델 능력과 평가 메모리가 지배)이며, **속도 향상(speedup)은 하드웨어에 강하게 의존**한다. 예컨대 정사각 행렬곱(L1 P1)의 speedup은 Blackwell 1.61× 대 H100 0.83×로, H100의 강력한 cuBLAS 기준선 때문에 생성 Triton 커널이 상대적으로 불리하다. 따라서 speedup의 절대 비교는 동일 하드웨어 내에서만 유효하며, Blackwell sm_120의 차별화는 정답률이 아닌 속도 · 실전 커널(FlashInfer) 축에서 추구해야 한다.

5.8 초대형 추론 모델의 극단 양자화 실험: 모델 크기 대 가중치 정밀도

"더 큰 모델이면 커널 생성이 나아지는가"를 검증하기 위해, 공개 최대급 추론 모델인 GLM-5.2(MoE, BF16 기준 약 1.5TB)를 H100 7-GPU 노드(각 80GB)에 극단 양자화로 적재하여 동일 프로토콜(best-of-1, 반복 3)로 평가하였다. 서빙은 llama.cpp의 레이어 분할 파이프라인 병렬(GPU 6장, 마지막 1장은 평가 전용)로 수행하였고, 2-bit(UD-Q2_K_XL, 254GB)와 서빙 예산(480GB) 내 최대 정밀도인 4-bit(UD-Q4_K_XL, 467GB)를 비교하였다(Table 12).

결과는 명확한 부정적 증거(negative evidence)를 제공한다. 2-bit GLM-5.2는 L1에서 0.10에 그쳤고 **L2/L3/FlashInfer에서는 단 한 문제도 해결하지 못했다**(레벨당 약 8.5시간, 총 21.5시간 소요). 세 가지 함의가 있다. 첫째, **양자화 손실은 문제 복잡도에 비례해 치명적**이다. 단일 연산(L1)에서는 일부 생존하던 생성 능력이, 여러 연산의 융합(L2)과 전체 모델(L3)처럼 긴 정밀 토큰 시퀀스가 필요한 과제에서 완전히 붕괴한다. 둘째, **가중치 정밀도를 희생한 모델 크기는 도메인 특화를 이기지 못한다**. 파라미터 수가 두 자릿수 배 큰

Table 12. GLM-5.2 양자화 수준별 정답률(Fast-0, H100, best-of-1 · 반복 3). 동일 조건의 Qwen-30B · AutoTriton-8B(Table 10)를 참조로 병기.

레벨	GLM 2-bit	GLM 4-bit	Qwen-30B	AT-8B
L1 (100)	0.10	0.23[§]	0.27	**0.47**
L2 (100)	0.00	–	0.05	**0.10**
L3 (50)	0.00	–	0.04	**0.06**
FlashInfer (8)	0.00	–	0.00	0.00

[§]표면 정답률. 문제당 2시간 한도 내 완주한 24문제 기준으로는 **0.96**(23/24)이며, 나머지 76문제는 저속 추론(초당 약 8토큰)으로 한도 초과(총 30시간 소요). L2 이하는 추정중.

일반 추론 모델(2-bit)이 8B Triton 특화 모델(BF16)의 절반에도 미치지 못하며, 이는 §5.7의 결론(특화 > 규모)을 극단 사례로 재확인한다. 셋째, 추론(reasoning) 모델 특유의 긴 사고 토큰(시도당 약 2,600 토큰)은 정확도 이득 없이 비용만 증가시켰다. 한편 4-bit 재실험은 이 붕괴가 실제로 양자화 손실 때문이었음을 확증한다: **시간 한도 내 완주한 문제 기준 정답률이 0.96(23/24)으로 극적으로 회복**되어 오픈 모델 중 최고 수준(상용 opus의 1.00에 근접)을 보였다. 그러나 4-bit의 저속 추론(2-bit의 절반 이하)으로 전체의 76%가 문제당 2시간 한도를 초과, 표면 정답률은 0.23에 그쳤다. 즉 병목이 *정밀도(2-bit)에서 처리량(4-bit)으로 이동*한 것으로, 이는 §6.5의 벽시계 제약 발견과 정확히 일치한다. 종합하면 제한된 GPU 예산에서 "초대형 모델을 낮은 정밀도로"는 품질이, "온전한 정밀도로"는 속도가 무너지므로, "중소형 모델을 온전한 정밀도로" 서빙하는 편이 커널 생성에 유리함을 실증한다.

6 논의

6.1 핵심 발견: 에러 전이 패턴

본 연구의 가장 중요한 발견은 LLM의 Triton 코드 생성 능력이 향상됨에 따라 에러 유형이 체계적으로 전이된다는 것이다. 이를 **에러 전이 가설(Error Transition Hypothesis)**로 정리한다: 모델 능력이 향상될수록 컴파일 에러(문법적 실패)에서 값 불일치(의미적 실패)로 지배적 에러 유형이 전이된다(Figure 5). gpt-4o-mini의 컴파일 에러율 71–82%에서 gpt-5.2의 7–26%로의 감소와, 동시에 값 불일치의 20%에서 59–72%로의 증가는 이 가설을 강하게 뒷받침한다. 이는 LLM이 Triton DSL의 문법을 학습하는 것은 비교적 용이하나, GPU 연산의 수치적 의미론(numerical semantics)을 이해하는 것은 본질적으로 어려운 과제임을 시사한다.

이 발견은 향후 LLM 기반 커널 생성 연구의 방향을 제시한다. 현재 대부분의 반복적 개선 전략은 에러 메시지를 피드백으로 제공하는 방식이나, 값 불일치 에러의 경우 에러 메시지만으로는 수정 방향을 결정하기 어렵다. 따라서 중간 결과 비교, 텐서 형상(shape) 추적, 수치 안정성 분석 등 보다 풍부한 피드백 메커니즘이 필요하다.

6.2 핵심 발견: 확률적 다양성의 우위

반복 횟수 ablation 연구에서 발견한 **확률적 다양성 원리(Stochastic Diversity Principle)**는 실용적으로 중요한 시사점을 제공한다. 4개 독립 실행의 합집합(28개)이 최선의 단일 실행(22개)을 27.3% 상회하며, 각 실행이 고유하게 해결하는 문제가 존재한다(Table 4). 이는 LLM의 코드 생성이 본질적으로 확률적(stochastic) 과정이며, 동일 문제에 대해서도 다른 경로의 해를 탐색할 수 있음을 의미한다.

이 원리의 실용적 귀결은 명확하다: 단일 모델로 깊이(depth) 있는 반복보다, 여러 독립적 시도를 통한 넓이(breadth) 우선 탐색이 더 비용 효율적이다. 이는 best-of-N 전략이나 다중 모델 캐스케이딩 전략의 이론적 근거를 제공하며, 실제로 gpt-4o+gpt-5.2 캐스케이딩이 2.44배 향상을 달성한 결과와 일맥상통한다.

6.3 비용 효율성 분석

본 실험 결과는 고비용 추론 모델이 반드시 우수한 성능을 보장하지 않음을 명확히 보여준다. gpt-5.2-pro는 gpt-5.2 대비 수 배 높은 API 비용과 긴 응답 시간에도 불구하고, 캐스케이딩 전략에서 단 1개의 추가 문제만 해결하였다. 코드 생성 태스크에서는 추론 깊이보다 코드 패턴에 대한 학습 범위가 더 중요한 요인이다.

실험 결과를 종합하면, gpt-4o와 gpt-5.2의 2단계 캐스케이딩이 비용 대비 최적의 전략이다. gpt-5.2-pro의 추가는 0.4%p의 미미한 향상만을 제공하므로, 비용 효율 관점에서 권장되지 않는다.

6.4 핵심 발견: 플래그십 수렴 현상

확장 실험에서 발견한 가장 흥미로운 현상은 gpt-5.5와 claude-opus-4-7이 처음 44문제에서 **정확히 동일한 37개 문제**를 해결했다는 점이다. 두 모델은 서로 다른 회사(OpenAI vs Anthropic)에서 독립적으로 개발되었음에도, P4만을 유일한 공통 실패로 공유하며 나머지 43문제에서 완벽하게 일치하는 성공/실패 패턴을 보였다.

이러한 **플래그십 수렴 현상(Flagship Convergence)**은 최상위 모델들이 일정 난이도 이하의 Triton 커널 생성 문제에 대해 사실상 동일한 능력 수준에 도달했음을 시사한다. 그러나 P45 이후 후반부에서 두 모델은 극적으로 분기하였다: gpt-5.5는 추가 0개, claude-opus-4-7은 추가 9개(P88–P100)를 해결하였다. 이 **후반부 회복 능력(late recovery)**이 claude-opus-4-7을 단일 모델 최고 성능(46/100)으로 이끈 핵심 차별화 요인이다.

또한, claude-opus-4-7 단일 모델(46/100)이 이전 3-모델 캐스케이딩 전략(43/100)을 초과한 것은 주목할 만하다. 이는 최신 세대 플래그십 모델의 발전이 다중 모델 전략의 필요성을 줄일 수 있음을 시사한다. 그러나 5개 모델 전체의 합집합(51/100)이 최선의 단일 모델보다 여전히 10.9% 높다는 점은, 확률적 다양성 원리가 최신 모델에서도 여전히 유효함을 확인시켜 준다.

Table 13. 동일 파이프라인에서 test-time 예산에 따른 Level 1 정답률. bo$N$=best-of-N, it$K$=반복 K회. 전수(100문제) 행은 Table 10, 정밀(P1–P20) 행은 §5.6 · §5.7의 측정이다. ¶완주한 38문제 기준(정답 29). 나머지 59문제는 문제당 시간 한도(2시간, 27회 시도 × 저속 서빙) 초과로 미완주 — 예산 확대의 실효가 벽시계 처리량에 의해 제약됨을 보여준다(총 22.9시간 소요).

예산	범위	Qwen-30B	AT-8B	opus-4-7
bo1·it3 (최소)	L1 전체	0.27	0.47	–
bo3·it9 (충분)	L1 P1–20	0.75	**0.95**	**1.00**
bo3·it9 (충분)	L1 전체 (A6000)	–	0.76¶	–

6.5 100% 정답률 연구들과의 격차 해부: 예산 통제 관점

KernelEvolve(100%) [35], CudaForge(97.6%) [59], QiMeng(L1–2 ~100%) [61] 등 최근 연구들이 KernelBench에서 사실상 만점을 보고하는 반면, Table 10의 본 연구 수치(L1 0.27–0.47)는 언뜻 큰 격차로 보인다. 그러나 본 연구의 통제 실험들은 이 격차가 모델 능력의 격차가 아니라 **대부분 test-time 계산 예산의 격차**임을 보여준다. Table 13은 동일 파이프라인에서 예산 축만 바꾼 정답률을 정리한 것이다.

세 가지 요인으로 격차를 분해할 수 있다. 첫째, **test-time scaling이 지배적 요인**이다. 같은 오픈소스 8B 모델(AutoTriton)이 최소 예산에서 0.47, 충분한 예산(best-of-3, 반복 9, 실패 규칙 누적)에서 0.95로 상승하여 100% 연구들과의 격차 대부분을 소거한다. L1 전체 100문제로의 확장 재현(A6000)에서도 완주 문제 기준 정답률이 0.37(최소 예산)에서 0.76으로 두 배가 되었으나, 동시에 전체의 59%가 문제당 시간 한도를 초과해 미완주하였다 – 즉 *예산 확대의 실효는 시도당 처리량(서빙 속도와 병렬 경쟁)이라는 벽시계 제약에 의해 한정*되며, 이는 100% 연구들이 고속 상용 API 혹은 대규모 서빙 인프라를 전제하는 이유를 설명한다. TritonBench에서도 동일 축이 Execution Accuracy를 약 2.3배 올렸다(§5.7). 100%를 보고하는 연구들은 예외 없이 무제한 트리 탐색(KernelEvolve), 프로파일링 동반 다회 수정(CudaForge), 다단계 메타 프롬프트와 수리 루프(QiMeng), 대규모 RL(Kevin [4], CUDA Agent [10]) 등 큰 시도 예산을 전제하며, 단일 시도 조건의 수치를 보고하지 않는다. 즉 Table 10는 능력의 상한이 아니라 *예산 통제된 하한(budget-controlled floor)*이다. 둘째, **평가 인프라와 프롬프트 결함**이 한때 정답률을 최대 0.30까지 잠식했으며(OOM 거짓 실패, 입력 dtype 계약 부재; §5.6), 수정 후에는 재현 가능하게 회복되었다. 셋째, 남는 잔차는 **모델 능력**이다. 충분한 예산에서도 Qwen-30B는 0.75에 머무나, Triton 특화 RL을 거친 8B 모델은 0.95로 상용 플래그십(1.00)에 근접한다. 요컨대 **상용 모델 없이도 "도메인 특화 오픈 모델 + 충분한 test-time 예산 + 건전한 평가 인프라"의 결합으로 만점권 정답률에 도달 가능**하며, 문헌의 100% 수치와 본 연구의 전수 스캔 수치는 서로 다른 예산 체제의 측정으로 이해해야 한다. 부수적으로, 검증 엄격성의 차이도 있다. 본 평가는 무작위 입력 5회 반복과 dtype/shape 계약을 요구하는 반면, 문헌에는 보상 해킹으로 부풀려진 수치가 보고된 사례 [27]가 있어, 예산과 검증 조건을 명시하지 않은 정답률 비교는 오도될 수 있다.

에이전트 구조 자체의 기여도 통제 실험으로 분리하였다. 100%를 보고한 공개 딥에이전트 KernelFalcon의 코드(KernelAgent)를 동일 오픈 모델(Qwen-30B, 로컬 서빙)로 재현한 결과, L1 P1–20 정답률은 본 파이프라인의 고예산 설정과 **동률(0.75)**이었고 실패 집합(ND/전치 행렬곱)도 일치하였다. 다만 벽시계 시간은 수 시간에서 약 1시간으로 크게 단축되었다(4-시드 병렬 탐색). 즉 **에이전트 구조는 탐색 효율을 높일 뿐 기반 모델의 능력 한계를 넘지 못하며**, 같은 구조가 상용 모델(gpt-5.2)로는 해당 실패 문제(예: 전치 both)를 1–2라운드에 해결한 것과 대조된다 – 100% 시스템의 잔여 격차는 결국 기반 모델 능력이 채운다. 이를 재확인하기 위해 더 강한 오픈 모델을 같은 구조에 투입하였다: GLM-5.2(4-bit)를 H100 8-GPU에 생성 전용으로 서빙하고 검증을 로컬 GPU로 분리한 구성에서 KernelAgent×GLM-5.2는 **P1–20 전 문제를 해결(20/20, 1.00)**하여 상용 플래그십(opus, 1.00)과 동률에 도달하였다. Qwen이 두 구조 모두에서 실패한 3D/4D 및 전치 행렬곱 5문제를 포함하며, 1차 실행에서 남았던 2건(시간 한도 초과 1건, 추론 출력의 코드 블록 파싱 실패 1건)도 시간 한도 연장 재시도에서 즉시 해결되어 실패가 모델 능력이 아닌 운영 요인이었음이 확인되었다. 즉 *구조(효율) × 모델 능력(상한)의 곱*이 정답률을 결정하며, **충분히 강한 오픈 모델과 공개 에이전트 구조의 결합만으로 상용 모델 없이 만점에 도달할 수 있다**. 이 결합을 L1 전체(100문제)로 확장 재현하는 실험을 진행 중이며(생성=H100 8-GPU GLM-5.2 4-bit, 검증=로컬 Pro 6000, 문제당 2시간 한도, 두 문제 동시 실행으로 8슬롯 포화), 그 과정에서 에이전트 프레임워크의 응답 토큰 상한(기본 8192)이 GLM의 추론 출력(5–7K 토큰)을 코드 블록 출력 이전에 절단하여 정답 가능한 문제를 "코드 블록 없음"으로 거짓 실패시키는 현상을 발견하였다. 이 상한을 13000으로 상향하자 해당 구간(P40–49)의 정답률이 패치 이전 구간(P21–39, 0.37) 대비 약 두 배인 0.78로 회복되어, **서빙 컨텍스트 길이뿐 아니라 프레임워크의 응답 토큰 예산 또한 reasoning 모델의 평가를 왜곡하는 은닉 변수**임이 드러났다. 나아가 커널 생성에 특화된 RL 오픈 모델 Dr.Kernel-14B [36](14B, KernelGYM에서 멀티턴 RL로 학습)을 동일 KernelAgent 구조(best-of-4)에 이식하여 L1 P1–20을 평가한 결과 13/20(0.65)로, 같은 구조의 Qwen-30B(0.75)나 GLM-5.2(1.00)보다 오히려 낮았다. 이는 **RL 특화 모델의 이점이 자신의 훈련 하니스(멀티턴 STTS · 프로파일링 보상)에 최적화되어 있어, 서로 다른 에이전트 구조(4-시드 병렬 탐색)로 이식할 때 부분적으로만 전이됨**을 시사한다 – 즉 정답률은 모델 능력뿐 아니라 *평가 하니스와 훈련 하니스의 정합성*에도 의존하며, 이는 서로 다른 시스템에서 보고된 정답률을 직접 비교할 때 주의가 필요함을 다시 뒷받침한다. 마지막으로, 본 절의 핵심 주장인 "test-time 예산이 정답률을 지배한다"를 제3자 시스템에서 독립적으로 교차 검증하기 위해 Dr.Kernel의 공개 코드(KernelGYM)와 released 14B 모델로 순차 test-time scaling을 완전 재현하였다: L2 100문제에서 유효 턴 수를 1에서 14로 늘리자 전체 기준 Fast@1.2가 0.17에서 0.77로 *단조 증가*하였고(3턴 기준 0.24의 3.2배), best-of-history Fast@1은 0.86에 도달하여 논문 보고치(0.81)를 상회하였다 – 동일 모델 · 동일 문제에서 오직 추론 예산만으로 성능이 3배 이상 갈리는 이 곡선은,

예산 명시 없는 정답률 비교가 왜 무의미한지를 저자 독립적 재현으로 보여준다. L1 전체 확장의 실행분(77문제) 정답률은 0.48이었으며, matmul/reduction/elementwise 계열에서는 높은 반면 합성곱(conv/conv_transposed) · 순차 스캔(cumsum) 계열에서 급격히 낮았다. 특히 conv 계열의 실패 원인을 절단(예산)과 능력으로 분리하기 위해, 서버를 슬롯 컨텍스트 32K(`--parallel 4`)로 재구성하고 응답 토큰 상한을 24000으로 올려 절단을 완전히 제거한 뒤 대표 conv 3문제를 재시도하였다. 그 결과 **절단은 사라졌으나(코드 블록 정상 출력) 세 문제 모두 refinement 예산 내 정답 수렴에 실패**하여, conv 계열의 잔여 실패는 토큰 예산이 아니라 **거대 텐서의 정확한 인덱싱을 요구하는 순수 모델 능력의 한계**임이 확인되었다. 즉 예산성 결함을 모두 제거한 뒤 남는 격차가 곧 기반 모델 능력이라는 본 절의 주장이 conv에서 이중으로 실증된다.

이 관점은 실무 지침으로 이어진다. 정답률 벤치마크 결과를 읽을 때는 (1) 시도 예산(best-of-N, 반복 수), (2) 검증 강도(무작위 입력 횟수, 허용 오차, 계약 검사), (3) 평가기 자원 여유(서빙과 평가의 메모리 분리), (4) 서빙 컨텍스트 길이(짧으면 반복 수정 프롬프트가 잘려 refinement가 무력화됨), (5) 프레임워크의 응답 토큰 상한(reasoning 모델은 코드 이전에 수천 토큰의 사고 과정을 소비하므로 통상 8K 상한은 코드 블록 출력 전 절단을 유발한다)을 함께 보고해야 하며, 본 연구는 이 축들을 명시한 예산 통제 비교를 제공한다는 점에서 문헌을 보완한다.

6.6 한계 및 향후 연구

본 연구는 몇 가지 한계를 가진다. 첫째, 초기 실험에서 오픈소스 모델(Qwen 2.5 Coder 32B)의 평가는 평가기의 GPU 메모리 경합 결함으로 인해 정답률이 과소평가되었다(§5.6). 이 결함을 수정한 뒤 Level 1 부분집합 재측정에서 정답률이 회복되었으며(Table 9), 충분한 test-time 예산(best-of-3, 반복 9, 실패 규칙 누적)에서의 Level 1 전체(100문제) 재평가는 현재 진행 중으로 Table 13의 부분집합 결과(AutoTriton-8B 0.95)가 전체에서도 유지되는지 확인할 예정이다. 마찬가지로 KernelAgent×GLM-5.2 조합의 Level 1 전체 확장 재현에서 실행분(77문제) 정답률 0.48을 얻었으며(P1–20 20/20), 미실행 conv 문제를 포함한 전수 상한 확정은 향후 과제로 남긴다 — conv 계열은 절단을 제거한 통제 재시도에서도 실패하여 현 오픈 모델(GLM-5.2 4-bit)의 능력 한계로 확인되었다. Level 2/3의 고예산 전면 재평가는 향후 과제로 남긴다. 둘째, 하드웨어 교차 평가를 Blackwell · H100 · A6000 세 아키텍처로 확장하여 정답률의 하드웨어 독립성을 확인하였으나(§5.7), speedup은 기준선(cuBLAS 등)이 하드웨어마다 달라 동일 하드웨어 내 상대 비교로 한정된다. 셋째, 확장 실험에서 Anthropic의 claude-opus-4-7을 포함하였으나, Gemini 등 다른 LLM 제품군과의 비교가 부재하며, 확장 실험은 Level 1에 한정되어 있다.

향후 연구 방향으로 다음을 제안한다.

1. **오픈소스 모델 재평가**: 수정된 파이프라인으로 Qwen, DeepSeek-Coder, CodeLlama 등의 오픈소스 모델을 공정하게 재평가한다. 특히 TritonRL [53]과 AutoTriton [32]에서 보인 RL 기반 미세 조정(fine-tuning) 접근과의 비교를 수행한다.
2. **값 불일치 해결 전략**: 에러 전이 가설에 기반하여, 값 불일치 에러에 특화된 피드백 메커니즘을 설계한다. 구체적으로 중간 텐서 값 비교, 수치 안정성 분석, 그리고 TritonForge [29]에서 제안된 프로파일링 기반 최적화를 통합할 수 있다.
3. **하이브리드 접근**: LLM 기반 초기 코드 생성과 Mirage [54]와 같은 수퍼옵티마이저(superoptimizer) 기반 후처리를 결합하는 하이브리드 파이프라인을 탐구한다.
4. **RAG 기반 패턴 학습**: Retrieval-Augmented Generation을 통해 기존의 성공적인 Triton 커널 패턴을 LLM의 문맥(context)에 주입하여, 특히 Level 3 수준의 복잡한 커널 생성 능력을 향상시킨다.

7 결론

본 논문은 LLM 기반 Triton GPU 커널 자동 생성에 대한 체계적 평가를 통해, 단순한 성능 비교를 넘어 커널 코드 자동 생성의 근본적 특성에 관한 핵심 원리를 발견하였다.

첫째, **에러 전이 가설**을 제안하고 실증적으로 검증하였다. LLM 능력이 향상됨에 따라 컴파일 에러에서 값 불일치로 지배적 에러 유형이 전이되며, 이는 Triton DSL 문법 학습과 GPU 연산 의미론 이해 사이에 본질적 난이도 격차가 존재함을 시사한다. 확장 실험에서 claude-opus-4-7의 컴파일 에러율 0%와 값 불일치 65%는 이 가설을 더욱 강하게 뒷받침한다.

둘째, **확률적 다양성 원리**를 발견하였다. 반복의 깊이보다 독립적 생성의 다양성이 더 효과적이며, 5개 모델 합집합(51/100, 51%)이 최선의 단일 모델(46/100)보다 10.9% 높은 결과는 최신 세대 모델에서도 이 원리가 유효함을 확인시켜 준다.

셋째, **플래그십 수렴 현상(Flagship Convergence)**을 발견하였다. gpt-5.5와 claude-opus-4-7이 처음 44문제에서 정확히 동일한 37개 문제를 해결하였으며, claude-opus-4-7은 후반부 회복 능력을 통해 단일 모델 최고 성능(**46/100, 46%**)을 달성하였다. 이는 이전 3-모델 캐스케이딩 전략(43/100)을 단일 모델로 초과한 최초의 결과이다.

넷째, 추론 특화 모델(gpt-5.2-pro)이 코드 생성 과제에서 비효율적임을 정량적으로 입증하였다. 한편, Level 1에서 49%의 문제가 여전히 미해결이며, 특히 값 불일치가 지배적 병목이라는 발견은 GPU 연산의 의미적 이해 향상이 차세대 핵심 과제임을 명확히 한다.

끝으로, **평가 인프라 자체가 측정 결과를 좌우할 수 있음**을 발견하였다. GPU 메모리 경합으로 인한 거짓 실패를 제거하고 입력 dtype 계약, 정확성 게이팅, 문맥 초기화 재시도를 도입한 결과, Level 1 부분집합(20문제)에서 claude-opus-4-7이 100% 정답률을 달성하였다. 또한 프로파일링 피드백 ablation을 통해, 정확성이 선결되지 않은 상태에서의 성능 최적화 신호(ncu)는 효과가 없음을 확인하였다. 이는

LLM 커널 생성 연구에서 모델 능력과 평가 인프라를 분리하여 측정하고, 정확성을 성능에 선행시키는 설계가 중요함을 강조한다.

본 연구의 프레임워크와 발견은 LLM 기반 GPU 커널 생성 연구 커뮤니티에 체계적인 평가 방법론과 향후 연구 방향을 제시한다.

References

[1] Anonymous. 2026. TritonGym: A Benchmark for Agentic LLM Workflows in Triton GPU Code Generation. In *Submitted to the International Conference on Learning Representations (ICLR)*. Code and leaderboard available at https://github.com/yil384/TritonGym.

[2] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burber, et al. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *ASPLOS*.

[3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021).

[4] Carlo Baronio, Pietro Marsella, Ben Pan, Simon Guo, and Silas Alberti. 2025. Kevin: Multi-Turn RL for Generating CUDA Kernels. arXiv:2507.11948 [cs.LG]

[5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. In *arXiv preprint arXiv:2107.03374*.

[6] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Q. Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *USENIX Symposium on Operating Systems Design and Implementation*. https://api.semanticscholar.org/CorpusID:52939079

[7] Kun Cheng, Songshuo Lu, Sicong Liao, Tankun Li, Yafei Zhang, et al. 2026. MusaCoder: Native GPU Kernel Generation with Full-Stack Training on Moore Threads GPUs. arXiv:2606.04847 [cs.CV]

[8] Yee Hin Chong, Jiaming Wu, Youhui Zhang, and Peng Qu. 2026. Towards Feedback-to-Plan Decisions for Self-Evolving LLM Agents in CUDA Kernel Generation. In *Proceedings of the International Conference on Machine Learning (ICML)*. arXiv:2605.26720 [cs.AI]

[9] Giuseppe Crupi, Rosalia Tufano, and Gabriele Bavota. 2026. Improving Code Generation via Small Language Model-as-a-judge. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering (ICSE)*.

[10] Weinan Dai, Hanlin Wu, Qiying Yu, Huan-ang Gao, et al. 2026. CUDA Agent: Large-Scale Agentic Reinforcement Learning for High-Performance CUDA Kernel Generation. arXiv:2602.24286 [cs.LG]

[11] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691* (2023). arXiv:2307.08691

[12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS* (2022).

[13] Juncheng Dong, Yang Yang, Tao Liu, Yang Wang, Feng Qi, Vahid Tarokh, Kaushik Rangadurai, and Shuang Yang. 2025. STARK: Strategic Team of Agents for Refining Kernels. arXiv:2510.16996 [cs.LG]

[14] Weihua Du, Jingming Zhuo, Yixin Dong, Andre Wang He, Weiwei Sun, Zeyu Zheng, Manupa Karunaratne, Ivan Fox, Tim Dettmers, Tianqi Chen, Yiming Yang, and Sean Welleck. 2026. AdaExplore: Failure-Driven Adaptation and Diversity-Preserving Search for Efficient Kernel Generation. arXiv:2604.16625 [cs.LG]

[15] Jingzhi Gong, Vardan Voskanyan, Paul Brookes, Fan Wu, Wei Jie, Jie Xu, Rafail Giavrimis, Mike Basios, Leslie Kanthan, and Zheng Wang. 2025. Language Models for Code Optimization: Survey, Challenges and Future Directions. arXiv:2501.01277 [cs.SE]

[16] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y.K. Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).

[17] Siqi Guo, Ming Lin, and Tianbao Yang. 2026. DRTriton: Large-Scale Synthetic Data Driven Reinforcement Learning for Triton Kernel Generation. arXiv:2603.21465 [cs.LG]

[18] Yaochen Han, Ke Fan, Hongxu Jiang, Wanqi Xu, Weiyu Xie, Runhua Zhang, Chenhui Zhu, and Yixiang Zhang. 2026. EGG: An Expert-Guided Agent Framework for Kernel Generation. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*.

[19] Siva Kumar Sastry Hari, Vignesh Balaji, Sana Damani, Qijing Huang, and Christos Kozyrakis. 2026. Improving Efficiency of GPU Kernel Optimization Agents using a Domain-Specific Language and Speed-of-Light Guidance. arXiv:2603.29010 [cs.LG]

[20] Zixuan Huang, Da Chen, Kecheng Huang, Lihao Yin, Xing Li, Huiling Zhen, Mingxuan Yuan, and Zili Shao. 2026. Kernel Foundry: A Diagnosis-driven Evolutionary Kernel Optimizer with Multi-Experts. arXiv:2605.30359 [cs.NE]

[21] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, et al. 2024. Qwen2.5-Coder Technical Report. *arXiv preprint arXiv:2409.12186* (2024).

[22] Jaber Jaber and Osama Jaber. 2026. AutoMegaKernel: A Statically-Checked Agent Harness for Self-Retargeting Megakernel Synthesis. arXiv:2606.09682 [cs.LG]

[23] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR* (2024).

[24] Sathvik Joel, Jie JW Wu, and Fatemeh H. Fard. 2024. A Survey on LLM-based Code Generation for Low-Resource and Domain-Specific Programming Languages. *arXiv preprint arXiv:2410.03981* (2024). arXiv:2410.03981

[25] David B. Kirk and Wen-mei W. Hwu. 2016. *Programming Massively Parallel Processors: A Hands-on Approach* (3rd ed.). Morgan Kaufmann.

[26] Hyunho Kwon, Sanggyu Shin, Ju Min Lee, Hoyun Youm, Seungbin Song, Seongho Kim, Hanwoong Jung, Seungwon Lee, and Hanjun Kim. 2026. Compiler-Runtime Co-operative Chain of Verification for LLM-Based Code Optimization. In *Proceedings of the 2026 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*.

[27] Robert Tjarko Lange, Aaditya Prasad, Qi Sun, Maxence Faldor, Yujin Tang, and David Ha. 2025. *The AI CUDA Engineer: Agentic CUDA Kernel Discovery, Optimization and Composition*. Technical Report. Sakana AI. https://pub.sakana.ai/ai-cuda-engineer/paper/

[28] Kelun Lei, Hailong Yang, Huaitao Zhang, Xin You, Kaige Zhang, Zhongzhi Luan, Yi Liu, and Depei Qian. 2025. PRAGMA: A Profiling-Reasoned Multi-Agent Framework for Automatic Kernel Optimization. arXiv:2511.06345 [cs.DC]

[29] Haonan Li, Keyu Man, Partha Kanuparthy, Hanning Chen, Wei Sun, Sreen Tallam, Chenguang Zhu, Kevin Zhu, and Zhiyun Qian. 2025. TritonForge: Profiling-Guided Framework for Automated Triton Kernel Optimization. *arXiv preprint arXiv:2512.09196* (2025). arXiv:2512.09196

[30] Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, Haojie Wang, Jianrong Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025. TritonBench: Benchmarking Large Language Model Capabilities for Generating Triton Operators. In *Findings of the Association for Computational Linguistics (ACL)*. arXiv:2502.14752

[31] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. StarCoder: may the source be with you! *Transactions on Machine Learning Research* (2023).

[32] Shangzhan Li, Zefan Wang, Ye He, Yuxuan Li, Qi Shi, Jianling Li, Yonggang Hu, Wanxiang Che, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025. AutoTriton: Automatic Triton Programming with Reinforcement Learning in LLMs. *arXiv preprint arXiv:2507.05687* (2025). arXiv:2507.05687

[33] Xiaoya Li, Xiaofei Sun, Albert Wang, Jiwei Li, and Chris Shum. 2025. CUDA-L1: Improving CUDA Optimization via Contrastive Reinforcement Learning. arXiv:2507.14111 [cs.LG]

[34] Xuanzhe Li, Ziyan Weng, Zhiyu Zhu, and Junhui Hou. 2026. AgentCompile: An LLM-Guided Compiler for Direct CUDA Inference. arXiv:2606.07665 [cs.PL]

[35] Gang Liao, Carole-Jean Wu, Gaoxiang Liu, et al. 2026. KernelEvolve: Scaling Agentic Kernel Coding for Heterogeneous AI Accelerators at Meta. arXiv:2512.23236 [cs.DC]

[36] Wei Liu, Jiawei Xu, Yingru Li, Longtao Zheng, Tianjian Li, Qian Liu, and Junxian He. 2026. Dr. Kernel: Reinforcement Learning Done Right for Triton Kernel Generations. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*. KernelGYM environment and code available at https://github.com/hkust-nlp/KernelGYM.

[37] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. *ICLR* (2023).

[38] OpenAI. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).

[39] OpenAI. 2024. Learning to Reason with LLMs. https://openai.com/index/learning-to-reason-with-llms/.

[40] Anne Ouyang, Simon Zheng, Jiahui Xin, Charles Jiang, Abraham Luo, Tianjun Stojanov, Zicheng Bai, Kunle Chen, Trevor Darrell, Joseph E. Gonzalez, and Ion Stoica. 2024. KernelBench: Can LLMs Write GPU Kernels?. In *NeurIPS 2024 Workshop on Foundation Model Interventions*.

[41] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *SIGPLAN Not.* 48, 6 (jun 2013), 519–530. doi:10.1145/2499370.2462176

[42] Dezhi Ran, Shuxiao Xie, Mingfang Ji, Anmin Liu, Mengzhou Wu, Yuan Cao, Yuzhe Guo, Hao Yu, Linyi Li, Yitao Hu, Wei Yang, and Tao Xie. 2026. KernelBand: Steering LLM-Based Kernel Optimization via Hardware-Aware Multi-Armed Bandits. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*. Code available at https://github.com/TongmingLAIC/KernelBand.

[43] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2023. Code Llama: Open Foundation Models for Code. *arXiv preprint arXiv:2308.12950* (2023).

[44] Tara Saba, Anne Ouyang, Xujie Si, and Fan Long. 2026. CuTeGen: An LLM-Based Agentic Framework for Generation and Optimization of High-Performance GPU Kernels using CuTe. arXiv:2604.01489 [cs.LG]

[45] Taras Sereda, Burak Bartan, Ankita Nayak, Tom St. John, Natalie Serrino, and Zain Asgar. 2026. KForge: LLM-Driven Cross-Platform Kernel Generation for AI Accelerators. In *Machine Learning for Architecture and Systems Workshop (MLArchSys), co-located with ISCA*. arXiv:2606.02963

[46] Benjamin F. Spector, Simran Arora, Aaryan Singhal, Daniel Y. Fu, and Christopher Ré. 2024. ThunderKittens: Simple, Fast, and Adorable AI Kernels. *arXiv preprint arXiv:2410.20399* (2024). arXiv:2410.20399

[47] Annabelle Sujun Tang, Christopher Priebe, Lianhui Qin, and Hadi Esmaeilzadeh. 2026. COLT: Lightweight Multi-LLM Collaboration through Shared MCTS Reasoning for Model Compilation. arXiv:2602.01935 [cs.LG]

[48] Ali Tehrani, Yahya Emara, Essam Wissam, Wojciech Paluch, Waleed Atallah, Łukasz Dudziak, and Mohamed S. Abdelfattah. 2026. Fine-Tuning GPT-5 for GPU Kernel Generation. arXiv:2602.11000 [cs.LG]

[49] Shailja Thakur, Baleegh Ahmad, Zhenxing Fan, Hammond Pearce, Benjamin Tan, Ramesh Karri, Brendan Dolan-Gavitt, and Siddharth Garg. 2024. VeriGen: A Large Language Model for Verilog Code Generation. *ACM Transactions on Design Automation of Electronic Systems* 29, 3 (2024). doi:10.1145/3643681

[50] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (2019), 10–19.

[51] Arya Tschand, Muhammad Awad, Ryan Swann, Kesavan Ramakrishnan, Jeffrey Ma, Keith Lowery, Ganesh Dasika, and Vijay Janapa Reddi. 2025. SwizzlePerf: Hardware-Aware LLMs for GPU Kernel Performance Optimization. arXiv:2508.20258 [cs.LG]

[52] Jianghui Wang, Vinay Joshi, Saptarshi Majumder, Xu Chao, Bin Ding, Ziqiong Liu, Pratik Prabhanjan Brahma, Dong Li, Zicheng Liu, and Emad Barsoum. 2025. GEAK: Introducing Triton Kernel AI Agent & Evaluation Benchmarks. arXiv:2507.23194 [cs.LG]

[53] Jiin Woo, Shaowei Zhu, Allen Nie, Zhen Jia, Yida Wang, and Youngsuk Park. 2025. TritonRL: Training LLMs to Think and Code Triton Without Cheating. *arXiv preprint arXiv:2510.17891* (2025). arXiv:2510.17891

[54] Mengdi Wu, Xinhao Cheng, Shengyu Liu, Chunan Shi, Jianan Ji, Man Kit Ao, Praveen Velliengiri, Xupeng Miao, Oded Padon, and Zhihao Jia. 2025. Mirage: A Multi-Level Superoptimizer for Tensor Programs. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. arXiv:2405.05751

[55] Ruiyuan Xu, Shuoming Zhang, Guangli Li, Qiuchu Yu, Rui Zhang, Yangyu Zhang, Hao Qian, Chunwei Xia, Jiacheng Zhao, Chenxi Wang, Xiaobing Feng, Jingling Xue, and Huimin Cui. 2026. LEGO: An LLM-Enabled Hierarchical Optimizer for Tensor Computation Graphs with Structure-Aware Search and Compositional Synthesis. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*. Code available at https://github.com/mmt-at/LEGO.

[56] Yang Yu, Peiyu Zang, Chi Hsu Tsai, Haiming Wu, Yixin Shen, Jialing Zhang, Haoyu Wang, Zhiyou Xiao, Jingze Shi, Yuyu Luo, Wentao Zhang, Chunlei Men, Guang Liu, and Yonghua Lin. 2026. Towards Automated Kernel Generation in the Era of LLMs. *arXiv preprint arXiv:2601.15727* (2026). arXiv:2601.15727

[57] Genghan Zhang, Shaowei Zhu, Anjiang Wei, Zhenyu Song, Allen Nie, Zhen Jia, Nandita Vijaykumar, Yida Wang, and Kunle Olukotun. 2025. AccelOpt: A Self-Improving LLM Agentic System for AI Accelerator Kernel Optimization. arXiv:2511.15915 [cs.LG]

[58] Shuoming Zhang, Jiacheng Zhao, Qiuchu Yu, Chunwei Xia, Zheng Wang, Xiaobing Feng, and Huimin Cui. 2026. The New Compiler Stack: A Survey on the Synergy of LLMs and Compilers. arXiv:2601.02045 [cs.PL]

[59] Zijian Zhang, Rong Wang, Shiyang Li, Yuebo Luo, Mingyi Hong, and Caiwen Ding. 2025. CudaForge: An Agent Framework with Hardware Feedback for CUDA Kernel Optimization. arXiv:2511.01884 [cs.LG]

[60] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, et al. 2020. Ansor: Generating high-performance tensor programs for deep learning. In *14th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 20)*. 863–879.

[61] Xinguo Zhu, Shaohui Peng, Jiaming Guo, Yunji Chen, Qi Guo, Yuanbo Wen, Hang Qin, Ruizhi Chen, Qirui Zhou, Ke Gao, Yanjun Wu, Chen Zhao, and Ling Li. 2026. QiMeng-Kernel: Macro-Thinking Micro-Coding Paradigm for LLM-Based High-Performance GPU Kernel Generation. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*.